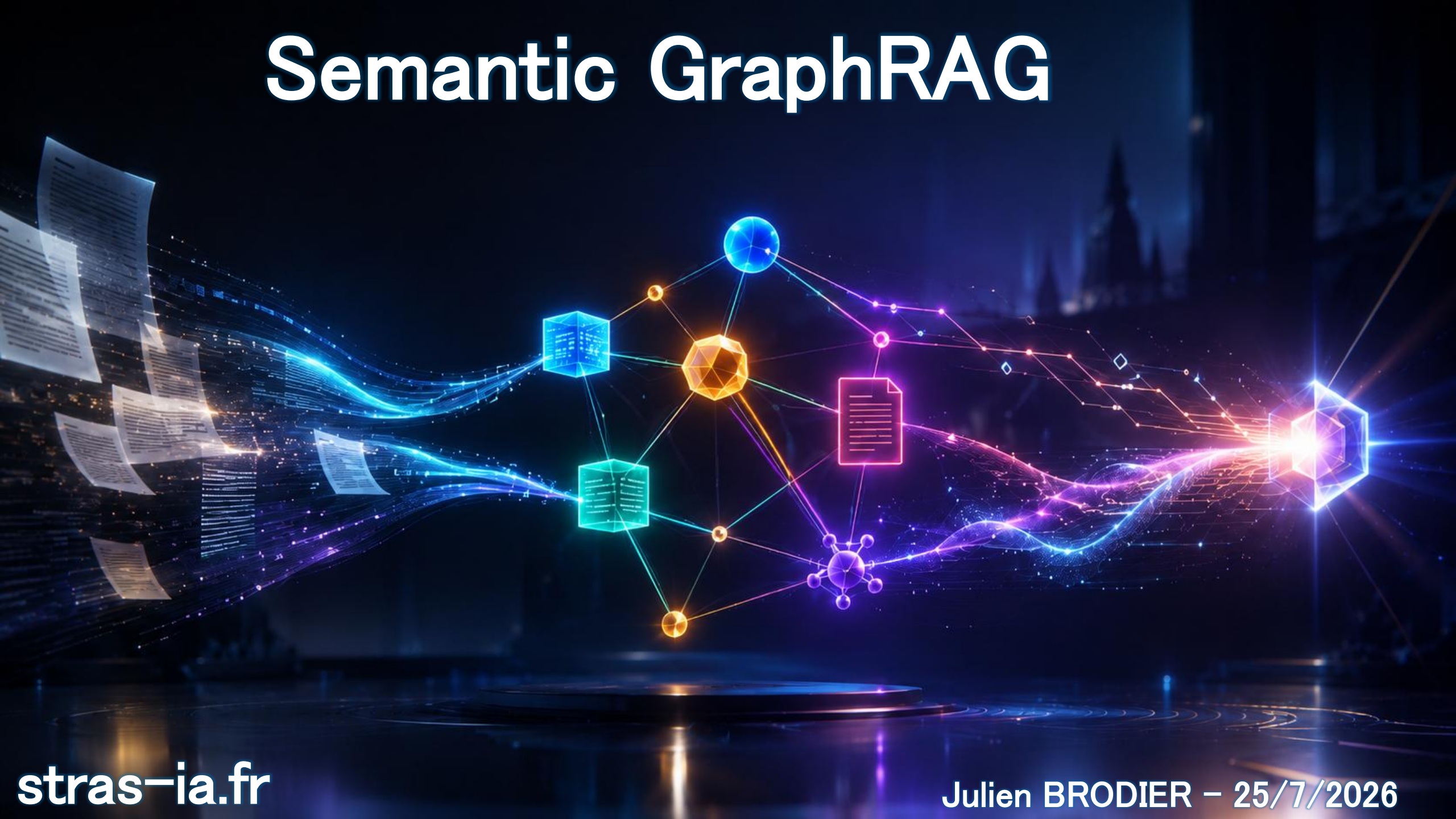
Semantic GraphRAG



stras-ia.fr

Julien BRODIER – 25/7/2026

From raw document to symbolic inference

0 **Why your RAG misses the point** - AI landscape, NLP, graphs & context engineering

1 **EXTRACT**
PDF => text
OCR · VLMs · Docling

2 **UNDERSTAND**
text => mentions
NER · RE · linking

3 **STRUCTURE**
mentions => graph
ontologies · granularity

4 **STORE**
graph => database
RDF vs LPG

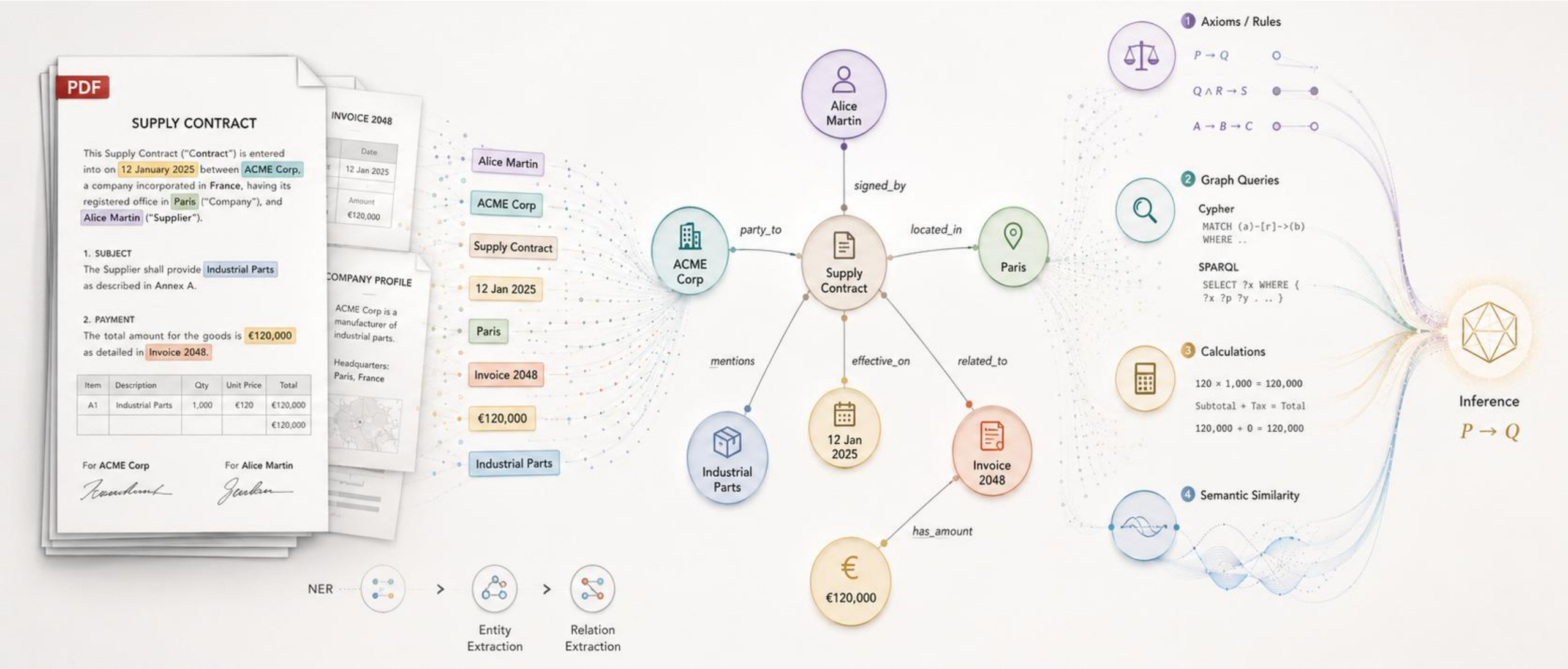
5 **QUERY**
question => answer
GraphRAG · rerankers

6 **REASON**
facts => new facts
axioms · inference · GNNs

PART 0

INTRO & AI LANDSCAPE

Semantic GraphRAG :
From raw PDF to symbolic inference



RAG = Search + Context Engineering

Retrieval-Augmented Generation: grounding an LLM's answers in your own documents, retrieved at query time.

RETRIEVE
Search

Classic information retrieval
- embeddings
- BM25
- grep
- metadata filters
- hybrid ranking

Recall is everything : a fact that isn't retrieved doesn't exist.

→

AUGMENT
Context engineering

Decide what enters the context window
- select,
- rerank,
- deduplicate,
- compress,
- order,
- structure

the evidence the model will actually see.

→

GENERATE
The easy part

LLM only verbalizes what it was given.

good search + well-built context
=> good answer

The LLM doesn't make your RAG good. Your search and your context engineering do

How Naive RAG Works

1 Indexing Phase (Offline)

Source Document

SERVICE AGREEMENT

1. **Purpose**
   This Agreement sets forth the terms and conditions under which the Service Provider agrees to provide services to the Client.
2. **Payment Terms**
   The Client shall pay all invoices within 30 days after the invoice date. Late payments may incur interest charges.
3. **Confidentiality**
   Both parties agree to keep confidential any non-public information disclosed in connection with this Agreement.
4. **Term and Termination**
   This Agreement begins on the Effective Date and continues for one year unless terminated earlier by either party.

...

Chunking

Overlap

Chunk 1
This Agreement sets forth the terms and conditions under which the Service Provider agrees to provide services to the Client.

Chunk 2
The Client shall pay all invoices within 30 days after the invoice date. Late payments may incur interest charges.

Chunk 3
Both parties agree to keep confidential any non-public information disclosed in connection with this Agreement.

Chunk 4
This Agreement begins on the Effective Date and continues for one year unless terminated earlier by either party.

⋮

Chunk 7
Governing Law. This Agreement shall be governed by and construed in accordance with the laws of the State of New York.

Embedding Model

e.g., all-MiniLM, text-embedding-3

Vectorization

[0.12, -0.45, 0.87, ...]
[0.09, -0.41, 0.82, ...]
[-0.31, 0.22, -0.71, ...]
[0.37, -0.12, 0.63, ...]
⋮
[-0.21, 0.74, -0.33, ...]

Vector Database (Embedding Index)

2 Retrieval & Generation Phase (Online)

User Query

What are the payment conditions of the contract?

Query Vectorization

Embedding Model (same as indexing)

[-0.08, -0.39, 0.81, ...]

Similarity

similarity(query, chunk)

Ranking

Top-K Results

1	Chunk 4	0.92
2	Chunk 2	0.86
3	Chunk 7	0.73
4	Chunk 3	0.41
5	Chunk 1	0.28

...

Retrieved Context (Top-K Chunks)

Chunk 4
This Agreement begins on the Effective Date and continues for one year unless terminated earlier by either party.

Chunk 2
The Client shall pay all invoices within 30 days after the invoice date. Late payments may incur interest charges.

Chunk 7
Governing Law. This Agreement shall be governed by and construed in accordance with the laws of the State of New York.

⋮

LLM Inference

Prompt (User Query)

Context (Retrieved Chunks)

Generated Answer

The contract requires payment within 30 days after invoice issuance.

Naïve RAG in 5 lines of code

```
# 1. Vectorize chunks  =>  matrix ( n_chunks : 1000 x dim : 768 )
chunks_vecs = numpy.array( [embed(c) for c in chunks] )

# 2. Vectorize the prompt
query_vec = numpy.array(embed(prompt))

# 3. Similarity (dot product / cosine if vectors are normalized)
scores = chunks_vecs @ query_vec

# 4. Ranking + top 10 extraction
top10 = [chunks[i] for i in numpy.argsort(scores)[::-1][:10]]

# 5. Augmented generation
answer = llm( f"Context: {top10} \n Question: {prompt}" )
```

Your RAG keeps missing the point

NAIVE RAG

Q: Who ultimately controls SCI Rhena?

A: Based on the documents, SCI Rhena was acquired by Muller Holding in 2026. Jean Muller, who serves as both notary and CEO of Muller Holding, ultimately controls the company.

✗ **fabricated deal** ✗ **two people merged**

Corpus, p.47: "SCI Rhena is 100% held by SARL Alsatia Invest." **— never retrieved.**

It hallucinates relationships

"X acquired Y" — a deal that never happened. Where did it come from? Next slide.

It merges entities

Two "Jean Muller"s — a notary and a CEO — collapsed into one person. Same string, same embedding.

It ignores black-and-white facts

The answer sits verbatim on page 47. Not similar to the question → never retrieved.

The problem isn't the LLM. It's how knowledge is represented.

Anatomy of a hallucinated deal

Q: "Who ultimately controls SCI Rhena?"

p.31 · RETRIEVED — Letter of intent

Chunk - rank : 1

"Muller Holding has expressed its intention to acquire all shares of SCI Rhena."

"This contemplated transfer, subject to the conditions precedent of Article 4, would take place in 2026"

Chunk - rank : 15

ERASED MODALITY

Modality erased: the condition is cut away
=> intention reads as fact.

p.58 · RETRIEVED — Deed of sale

Chunk - rank : 2

"By deed dated 12 May 2026, Muller Holding acquired all shares of SCI Beten."

FUSED DEALS

A real deal, but for a different company.

Fusion: same buyer, same year, same wording
=> Beten bleeds into Rhena.

p.47 · NOT RETRIEVED — Articles of assoc.

Chunk - rank : 38

"The capital of SCI Rhena is held in full (100%) by SARL Alsatia Invest."

MISSED CLAUSE

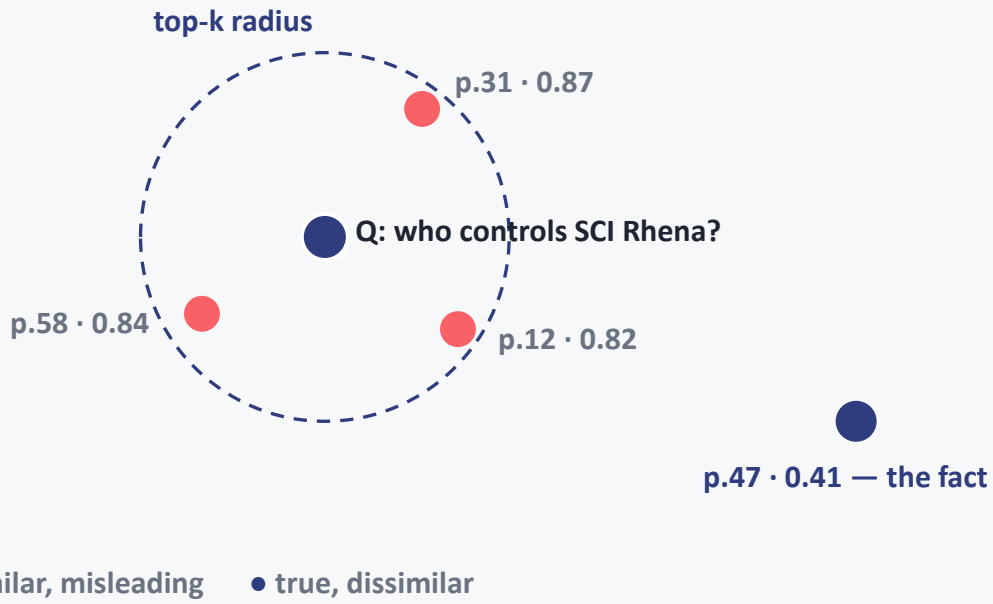
No acquisition ever happened,
but that absence is written nowhere.

wording never "similar" to the question.

Hallucination : "SCI Rhena was acquired by Muller Holding in 2026."

Similarity ≠ Factuality

EMBEDDING SPACE



TOP-K RETRIEVAL (cosine)

#1	0.87	*...intention to acquire all shares...*	✗
#2	0.84	*...acquired all shares of SCI Beten...*	✗
#3	0.82	*...governance rules of the SCI...*	✗

top-k cutoff (k = 5)

⋮

#38	0.41	*...capital held 100% by Alsatia Invest...*	✓

The fact - never enters the context window

"Who ultimately controls X?" is multi-hop: `(X)—held_by→(Y)—controlled_by→(Z)` - no single chunk contains the answer.

Vector search finds what is similar - not what is true. Facts need structure.

RAG vs GraphRAG

Same goal - give the LLM real context. One retrieves similar text; the other reasons over relationships.

	RAG retrieval by similarity	**GraphRAG** retrieval + graph reasoning
Data structure	Flat vector index	Knowledge graph - nodes + edges
Context	Isolated text chunks	Connected, relational context
Strengths	Fast, simple, any unstructured text	Multi-hop, relationships, global view
Limits	No reasoning · over-retrieval · static	Graph to build · 5-10× ingest cost
Best for	FAQ · doc Q&A · customer support	Multi-hop · supply chain · fraud · research

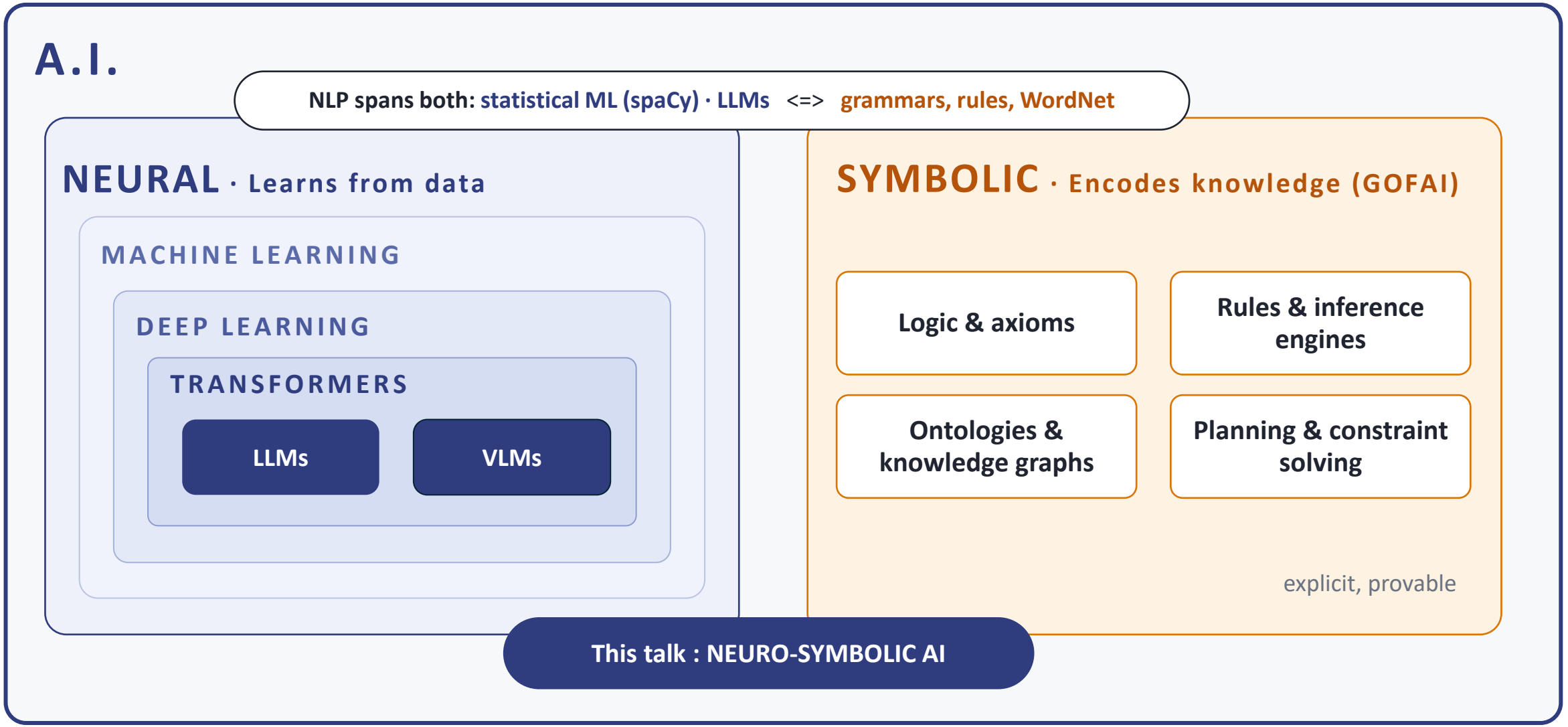
86% vs **32%** **(2.7×)**

Microsoft GraphRAG vs baseline RAG on multi-hop enterprise benchmarks

NOT A REPLACEMENT - AN EVOLUTION

The 2026 pattern is **hybrid routing**: a classifier sends simple queries to RAG, multi-hop & global ones to GraphRAG.

Natural Language Processing



neural for language

symbolic for truth

Application domains of NLP & RAG

NLP TASKS

Search | Classification | **Extraction** | Summarization | Translation | Conversational agents

WHAT AN ERROR MAY COST

Enterprise knowledge managmt.
decisions made on stale answers

Legal & notarial analysis
a missed clause

Corporate finance
a wrong owner

Compliance - GDPR & AI Act
a fine, an audit

Customer support
a wrong promise, at scale

Code assistants
a plausible bug

Knowledge Management & Context Engineering

KNOWLEDGE MANAGEMENT - TECHNIQUES

THE OLD

Taxonomies
Thesauri
Ontologies

THE NEW — 2013+

Embeddings
LLMs
Vector search

Can meet in the knowledge graph

CONTEXT ENGINEERING ≠ PROMPT ENGINEERING

Prompt engineering - wording the instructions.

Context engineering - deciding what enters the window:

Retrieve | **Rank** | **Compress** | **Structure**

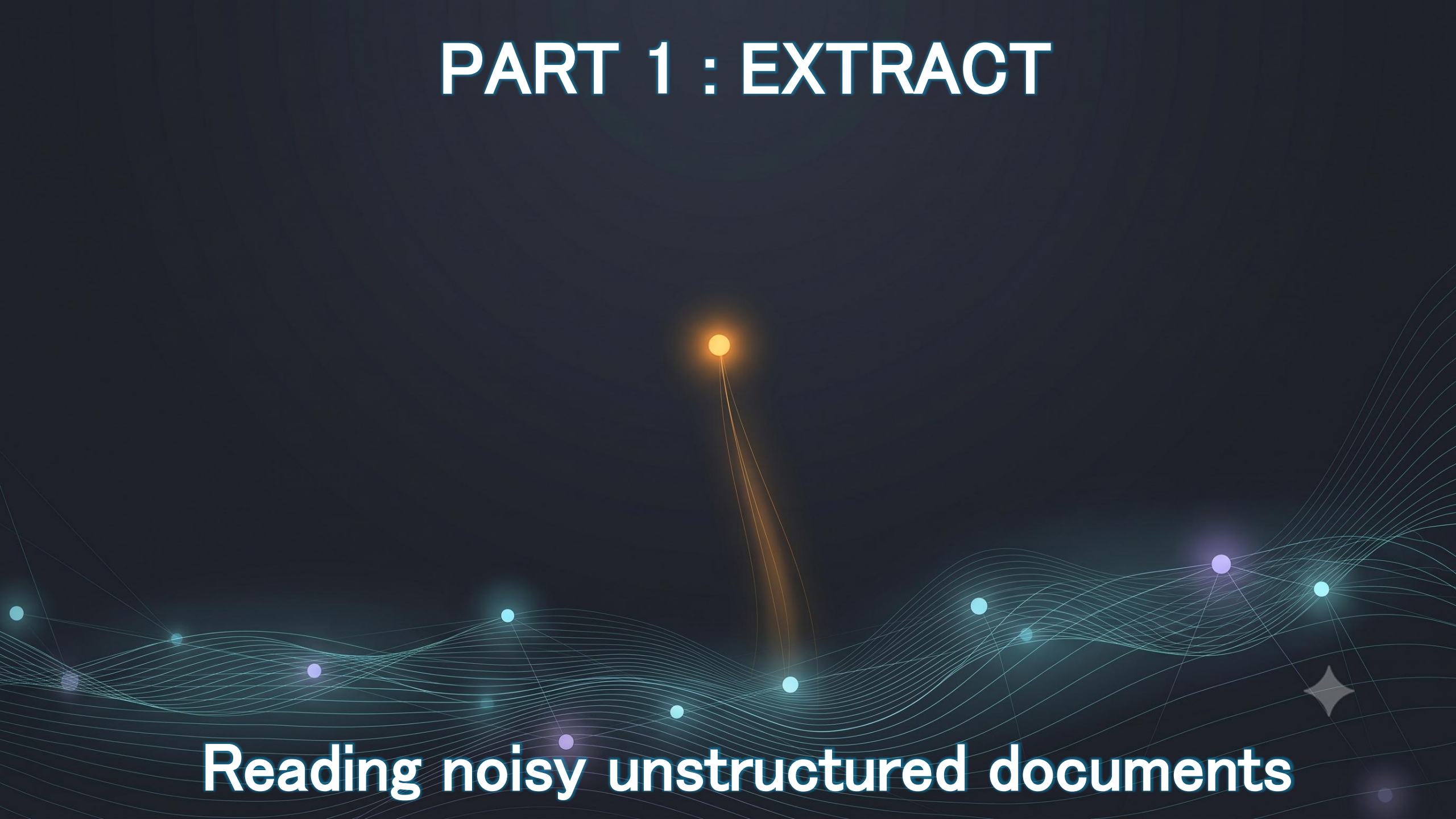
THE CONTEXT WINDOW - A SCARCE RESOURCE

system | **retrieved facts - your leverage** | **question**

The rest of this talk - six stages of context engineering done right: **Extract → Understand → Structure → Store → Query → Reason**

A knowledge graph is the ultimate context engineering tool: retrieve facts, not paragraphs.

PART 1 : EXTRACT



Reading noisy unstructured documents

Three ways to read a document

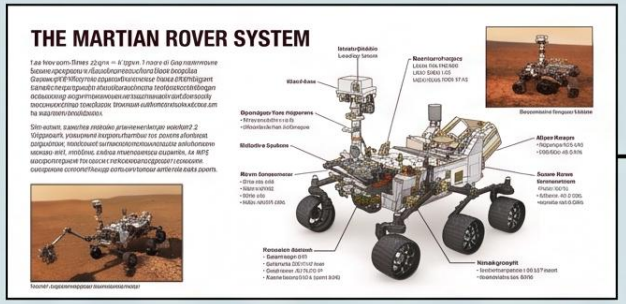
	Traditional OCR Tesseract · ABBYY	VLM Granite-Docling · Nanonets-OCR · Qwen-VL	Framework / wrapper Docling · Unstructured · Marker
Layout understanding	**Poor–medium**	**Strong**	**Strong** pipeline of specialized models
Tables	**Weak**	**Variable**	**Good** TableFormer-style models
Cost / latency	**Very low** CPU, ms/page	**High** GPU required	**Medium**
Determinism	**High** same input, same output	**Low** **can hallucinate text!**	**High-ish**
Handwriting & degraded scans	**Medium**	**Best**	**Depends on backend**

An OCR fails loudly.

A VLM fails silently : plausible text that was never on the page.

VLM – Vision Language Models

Input Sequence: Interleaved Text + Images



1. **Mixed-Media Document Page** (🖼)

"Please extract the main text paragraphs, key component specifications from the diagram, and the figure captions from this document page into a highly structured, well-formatted Markdown document, preserving reading order and table data."

2. **Structured Extraction Prompt**

- Document Images + Prompts
- Multiple Images
- Multi-turn Conversations
- Video (sampled frames)

Visual Tokens

Text Tokens

LLM Embedding Space

THE DECODER ONLY SEES A SINGLE SEQUENCE OF TOKENS.

Vision-Language Model (VLM)

Processor (e.g., CLIP/LLM)

Vision Encoder (e.g., ViT, CLIP/SigLIP)

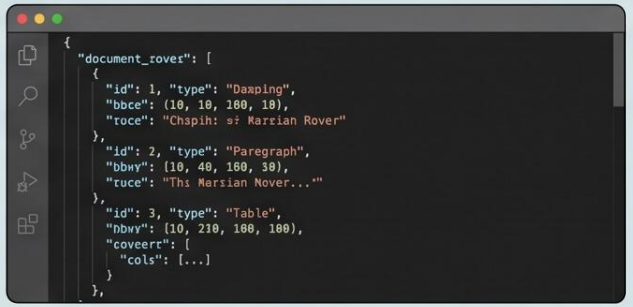
NOMINAL OUTPUT CASE: THE EXTRACTED AND STRUCTURED DOCUMENT TEXT (MARKDOWN)

1. **Canonical VLMs** (e.g., Qvanite-Docling)

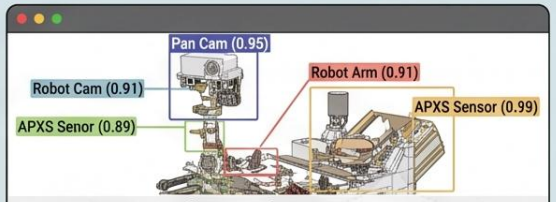
Canonical VLM Detailed Output Types

```
# Chapter 3: The Martian Rover System (TItle)

## Introduction (Section)

The Martion Rover, shown in Figure 1, is a mobile geological analysis platform...
[Intra paragraph]

## Rey System Specifications (Section)

The rover's primary analysis instruments are detailed in Table 3.1.

Table 3.1: Subsystem Mass and Power Distribution

| Subsystem       | Mass (kg) | Power (H) |
| --------------- | --------- | --------- |
| Mass Body       | 400       | 150       |
| Robotic Arm     | 150       | 300       |
| Mobility System | 150       | 90        |

## Subsystem Details (Section)

V#D Imaging Suite
(Description)

## Figure Caption (Section)

Figure 2.1: Clese-up details of the Alpha Particle X-ray Spectrometer (APXS) instrument.
```

Example 1: Structured Markdown (Hierarchy, Tables, Captions)



Example 2: Detailed Document Layout Metadata (JSON)



Extracted Description: [Figure 3.1] Deep analysis identifies and classifies operational subassemblies: (1) An active Articulated Robotic Arm subsystem, (2) Mast Camera suite, and (3) Alpha Particle X-ray Spectrometer sensor in foreground. Components are confidently parsed as distinct hierarchical elements.

Example 3: Hierarchical Figure/Diagram Parsing and Extracted Subsystem Descriptions

OmniDocBench: grading doc parsers

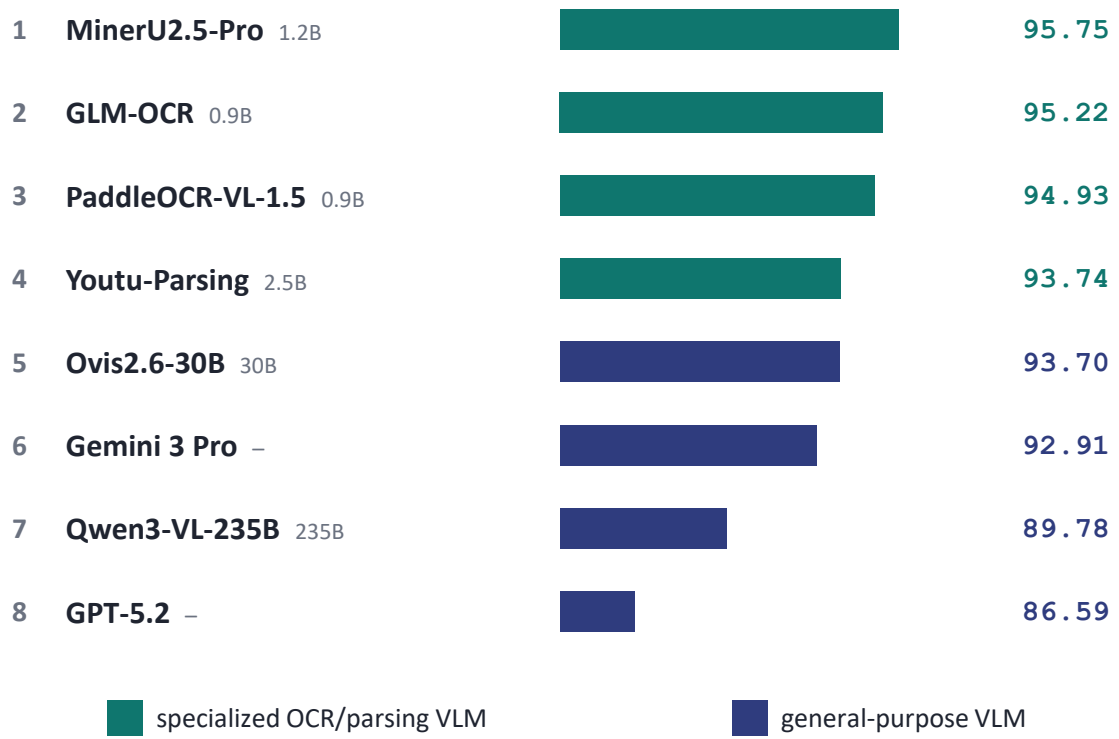
An open benchmark for turning real PDF pages into clean structured text and who's winning.

WHAT IT MEASURES

The task	PDF page => clean markdown: text, tables, formulas, in reading order
The data	1,651 real pages · 10 doc types · 5 languages · 5 layouts
5 tracks	end-to-end · text OCR · tables · formulas · layout + order
Metrics	Edit distance (text) · TEDS (tables) · CDM (formulas)
Overall	((1 − text edit) × 100 + table TEDS + formula CDM) / 3

CVPR 2025 · open dataset (Apache-2.0) · by OpenDataLab

TOP OF THE LEADERBOARD



Small specialized parsers top the board - a 1.2B model beats Gemini 3 Pro, GPT-5.2 and a 235B general VLM.

Which approach for which document?

ROUTE BY DOCUMENT TYPE

Born-digital PDF - text layer present	→	**Parser - no OCR at all** pdfplumber · PyMuPDF
Clean scans, standard layout	→	**Traditional OCR** Tesseract
Complex layout, forms, handwriting, degraded scans	→	**VLM** Qwen-VL · Granite-Docling
Mixed corpus, at scale	→	**Framework** Docling + VLM fallback

SILENT FAILURE MITIGATIONS

- Run OCR + VLM in parallel - diff the outputs
- Self-consistency: sample twice, compare
- Checksum validation: SIREN, IBAN, dates

REAL WORLD GOTCHAS

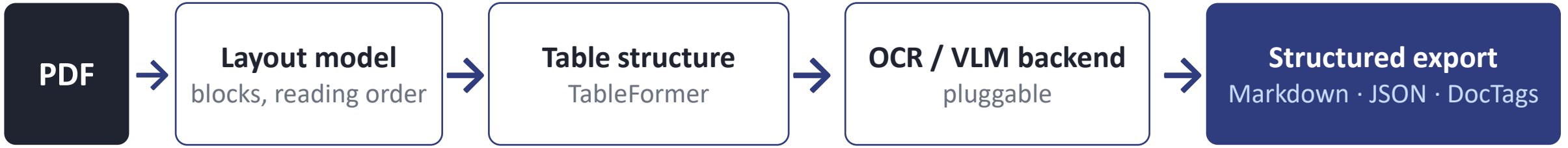
Encoding mismatch in the PDF text layer - accents mangled before you even parse

✓ `Société Générale`
✗ `SociÃ©tÃ© GÃ©nÃ©rale`

Multi-page tables: rows orphaned from their header at page breaks

Frameworks : Docling & friends

DOCLING PIPELINE



STRUCTURED OUTPUT

Headings, sections, tables survive parsing

=> chunk on structure, not on tokens

=> a table row keeps its header; a clause keeps its article

The chunker inherits the document's own logic

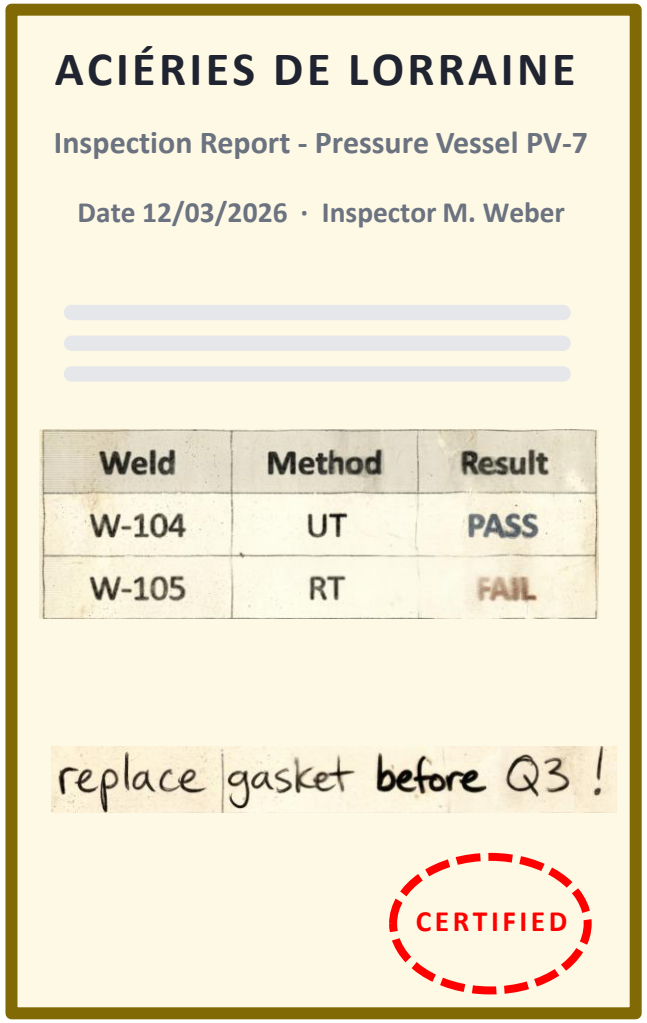
SERVING THE VLM BACKEND

Local, in-process - simple, dev-friendly, slow

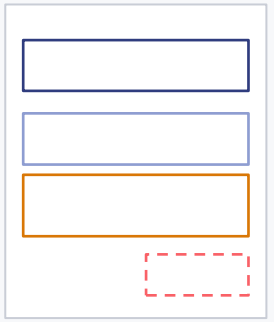
Inference server - vLLM · Ollama: batching, one GPU shared across pipeline workers

DocTags : markup made for LLMs: layout + content in one compact stream, cheaper than JSON, native to Granite-Docling

Docling pipeline on one document



1 · LAYOUT MODEL



Section header

Text blocks

Table

Stamp / note

blocks + reading order - per page

2 · TABLE STRUCT. TABLEFORMER

ched	ched	ched
fcel	fcel	fcel
fcel	fcel	fcel

predicts the grid as OTSL tokens + one **bbox** per cell.

Cell text is copied from the source - numbers are never generated.

header / cell / span structure, zero hallu'd amounts

3 · OCR / VLM BACKEND

`Inspection Report - PV-7` — text layer

`W-105 weld: RT re-test` — OCR

`"replace gasket before q3"` — VLM

cheapest reader per zone - routing, automated

4 · STRUCTURED EXPORT

```
<section_header>Inspection Report
PV-7
<otsl><ched>Weld<ched>Result<nl>
<fcel>W-105<fcel>FAIL<nl></otsl>
```

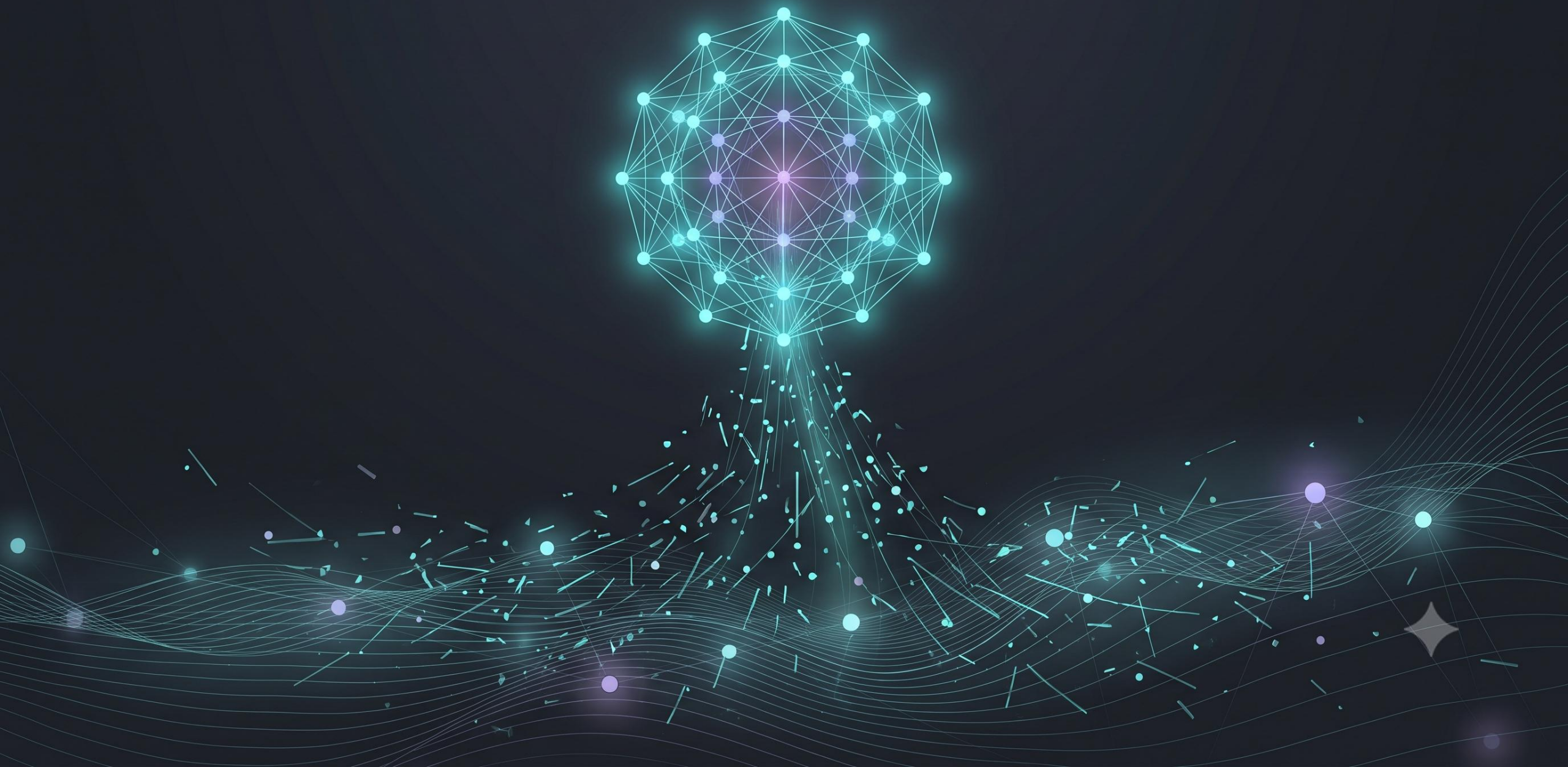
Markdown · JSON · DocTags

Every block keeps its page coordinates

Markdown output

```
# ACIÉRIES DE LORRAINE

## Inspection Report — Pressure Vessel PV-7

Date: 12/03/2026 · Inspector: M. Weber · Site: Hall B, Line 3

## 1. Scope

Periodic requalification of pressure vessel PV-7 (S355J2 steel, commissioned 2011) in accordance with the arrêté du 20 novembre 2017 (ESP). Visual inspection, hydrostatic test and non-destructive examination of the circumferential welds.

## 2. Test conditions

Hydrostatic test performed at 09:40 at 1.43 × PS (design pressure 16 bar), holding time 30 min. No visible deformation or leakage observed. Ambient temperature 14 °C.

## 3. Weld examination results

| Weld  | Method | Result |
|-------|--------|--------|
| W-104 | UT     | PASS   |
| W-105 | RT     | FAIL   |

Indication on W-105: linear indication, 6 mm, at the fusion line — exceeds acceptance criteria ISO 5817 level B. Repair and radiographic re-test required before return to service.

replace gasket before Q3 !

<!-- image -->
```

PART 2 : UNDERSTAND



Extracting Entities and Relations

PARSED DOC (MARKDOWN)

Inspection Report — **PV-7**

Vessel **PV-7** inspected on **12/03/2026** by **M. Weber** (**Bureau Veritas**), mandated by **Aciéries de Lorraine**

.

| Weld **W-105** | RT | FAIL |

Repair scheduled by **ACML** before Q3.

NER - TYPED MENTIONS

- Aciéries de Lorraine
- ACML
- Bureau Veritas
- M. Weber
- 12/03/2026
- PV-7 · W-105

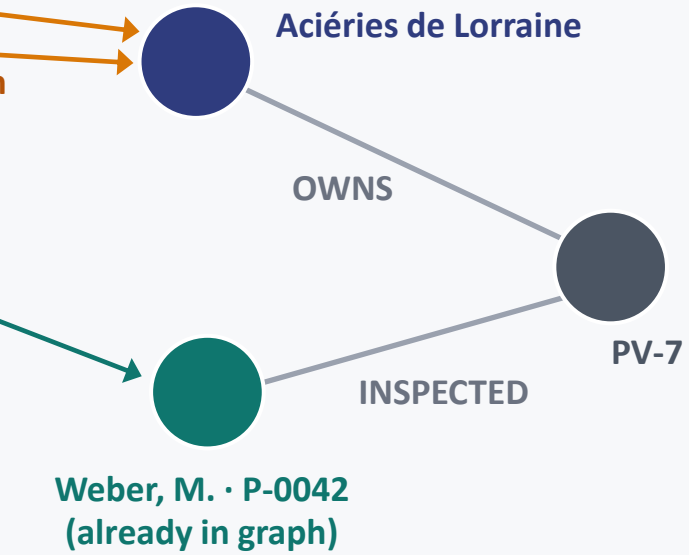
RE - TYPED RELATIONS

`(Weber)—INSPECTED→(PV-7)`

`(Weber)—WORKS_FOR→(Veritas)`

`(ACML)—OWNS→(PV-7)`

EXISTING KNOWLEDGE GRAPH



- company
- person
- date
- equipment

Resolution = merge co-referring mentions. ex: “Aciéries de Lorraine” + “ACML” => one node

Linking = attach to a node already in the graph

Named Entity Recognition : the contenders

	spaCy CNN / transformer pipelines	**GLiNER** zero-shot span classifier	**LLMs** prompted extraction
Custom entity types	**Retrain needed** annotated data + training run	**Zero-shot** **just name the type**	**Zero-shot** via prompt
Accuracy, standard types	**Very good**	**Good**	**Good but inconsistent**
Cost per 1M tokens	**~free** CPU	**Cheap** small GPU / CPU	**$$$**
Latency	**ms**	**ms - tens of ms**	**seconds**
Determinism / repeatability	**High**	**High**	**Low**
Structured output	**Native**	**Native** spans + character offsets	**JSON coercion needed** offset mapping is painful

volume => spaCy **new types => GLiNER** **relations => LLM**

NER with an LLM

You describe the entities in words; the model generates them as structured data.

INPUT: THE PROMPT

① **Instruction**

"Extract all named entities."

② **Type schema**

Company · Person · Deed · Notary

③ **Text**

"By deed n°2023-0512, Muller Holding acquired SCI Beten, signed before Maître Weber."

④ **Output format**

JSON [{text, type}]

GENERATE

LLM

reads context, then WRITES output token by token

OUTPUT: STRUCTURED JSON

```
[
  {"text": "Muller Holding", "type":"Company"},
  {"text": "SCI Beten", "type":"Company"},
  {"text": "Maître Weber","type":"Notary"},
  {"text": "n°2023-0512","type":"Deed"}
]
```

WHAT THE LLM UNLOCKS

- ✓ Zero-shot: any type, described in words
- ✓ Implicit & relational entities, not just literal spans
- ✓ Coreference + normalization in one pass
- ✓ Reasons about ambiguity from context

THE CATCH: IT GENERATES

- ✗ May hallucinate entities that aren't there
- ✗ Offsets unreliable => must align back to source
- ✗ Can drift from the schema - needs validation
- ✗ Slower & costlier per call than GLiNER / spaCy

spaCy

INPUT : TEXT + TRAINED PIPELINE

"By deed n°2023-0512, dated 12 May 2023, Muller Holding acquired all shares of SCI Beten."

en_core_web_sm

Fixed types, baked into the weights:

PERSON · ORG · DATE · GPE · LOC · DATE · PRODUCT · EVENT …

Contrast: GLiNER's types are strings

1 · TOKENIZER - RULES

```
By|deed|n°2023-0512|,|dated|12|May|
2023|,|Muller|Holding|acquired|all|
shares|of|SCI|Beten|.
```

"n°2023-0512" kept whole - exception table, not a model

deterministic rules: exceptions, punctuation, contractions (l', qu')

2 · TOK2VEC - EMBEDDINGS

- hash("Muller") → Bloom embedding
- CNN over window ±4 → context vector
- "Muller" knows "acquired" follows

no vocabulary file - hashing keeps it tiny; swap in a transformer (trf) for accuracy

3 · NER - TRANSITION-BASED

```
Muller        → B-ORG
Holding       → L-ORG
12 May 2023   → B/I/L-DATE
acquired      → O
n°2023-0512   → O   (unknown to the model)
```

greedy token-by-token decisions (BILUO) - ms, deterministic

4 · ENTITYRULER - RULES AGAIN

```
pattern: n°\d{4}-\d{4} → ACT_REF
deed n°2023-0512   ACT_REF   [3,19]
12 May 2023        DATE      [27,38]
Muller Holding     ORG       [40,54]
SCI Beten          ORG       [78,87]
```

Matcher & EntityRuler add what the weights can't know - offsets native

spaCy uses pre-trained statistical models to recognize entities.

GLiNER

INPUT : TEXT + LABELS

"By deed n°2023-0512, dated 12 May 2023, Muller Holding acquired all shares of SCI Beten."

Labels

- company
- date
- notarial act reference

no training data · no fine-tune

1 · ONE ENCODER, TWO INPUTS

`[ENT] company [ENT] date [ENT] act_ref [SEP] By deed n°2023-0512, dated 12 May 2023, Muller Holding acquired …`

one bidirectional pass (DeBERTa lineage) - labels become vectors too

2 · SPAN ENUMERATION

⟨By⟩ ⟨By deed⟩ ⟨deed n°2023-0512⟩ … ⟨Muller⟩ ⟨Muller Holding⟩ ⟨Holding acquired⟩ …

≈ 140 candidate spans (length ≤ 12)

every contiguous span gets one embedding from its boundary tokens

3 · SPAN × LABEL MATCHING

	company	date	act ref
Muller Holding	**.94**	.03	.02
12 May 2023	.02	**.97**	.08
deed n°2023-0512	.05	.11	**.89**
Holding acquired	.07	.02	.04

sigmoid per (span, label) - keep scores > τ, drop overlaps

4 · STRUCTURED OUTPUT

deed n°2023-0512	act_ref	[3,19]	.89
12 May 2023	date	[27,38]	.97
Muller Holding	company	[40,54]	.94
SCI Beten	company	[78,87]	.91

spans ⇒ native char offsets - nothing to re-locate, nothing hallucinated

GLiNER uses transformer models to extract entities based on type descriptions, not training data.

When to use what

spaCy — the workhorse

- High volume, stable entity schema
- ms latency, native offsets, CPU

legal doc pipelines · streaming

GLiNER — the schema-flexible

KG sweet spot

- New types = strings - zero training data
- “SIREN number” · “notarial act reference”

knowledge-graph extraction

LLM — the heavy artillery

- Relations, nested & ambiguous spans
- Low volume · judge / repair pass

arbitrate extractor disagreements

THE PRODUCTION PATTERN - EACH TOOL AT ITS LAYER

spaCy	**plumbing:** `tokens · sentences · lemmas (acquired=>acquire) · offsets`
GLiNER	**entities:** `⟨ Muller Holding: company ⟩ ⟨ SCI Beten: company ⟩`
LLM	**relations:** `( Muller Holding ) —ACQUIRED=> ( SCI Beten )`

cheap & deterministic first, expensive & generative last

Choosing the right approach

Scenario	Recommendation	Pipeline	Why
Fast indexing, common entities	spaCy only	`[spaCy]`	speed & simplicity
Domain-specific text	GLiNER + domain schema	`[GLiNER]`	custom types, solid accuracy
High-value documents	Full pipeline	`[spaCy → GLiNER → LLM]`	maximum accuracy
Mixed workload	Tiered pipeline	`[spaCy + GLiNER]`	speed / accuracy balance
Relations needed	**GLiNER + GLiREL**	`[GLiNERWithRelations]`	entities + relations, one pass

Typical latency, 500-word doc: **spaCy 10–50 ms** **GLiNER 50–100 ms (GPU)** **LLM 0.5–2 s**

Beyond NER: what graphs actually need

MENTIONS ≠ ENTITIES

Coreference (within a doc)
"the company" · "it" · "DURAND SARL" => one referent

Resolution (across docs)
"ACML" = "Aciéries et C.M. de Lorraine"

wrong => split companies, or two Jean Mullers merged

RELATIONS: (S, P, O) CANDIDATES

```
(ACML)   —OWNS=>       (PV-7)
(Weber)  —INSPECTED=>  (PV-7)
(Weber)  —WORKS_FOR=>  (Veritas)
```

candidates, not yet facts - validated against the ontology

Each carries doc · page · char offsets

ATTRIBUTES REGEX

```
SIREN 398 745 621 =>  ✓ Luhn
12/03/2026 =>   2026-03-12
"1,2 M€" =>   1 200 000 EUR
```

dates, amounts, identifiers
SpaCy matchers & regex, with checksum validation

deterministic · F1 ≈ 1.0 · free

PROVENANCE - NON-NEGOTIABLE

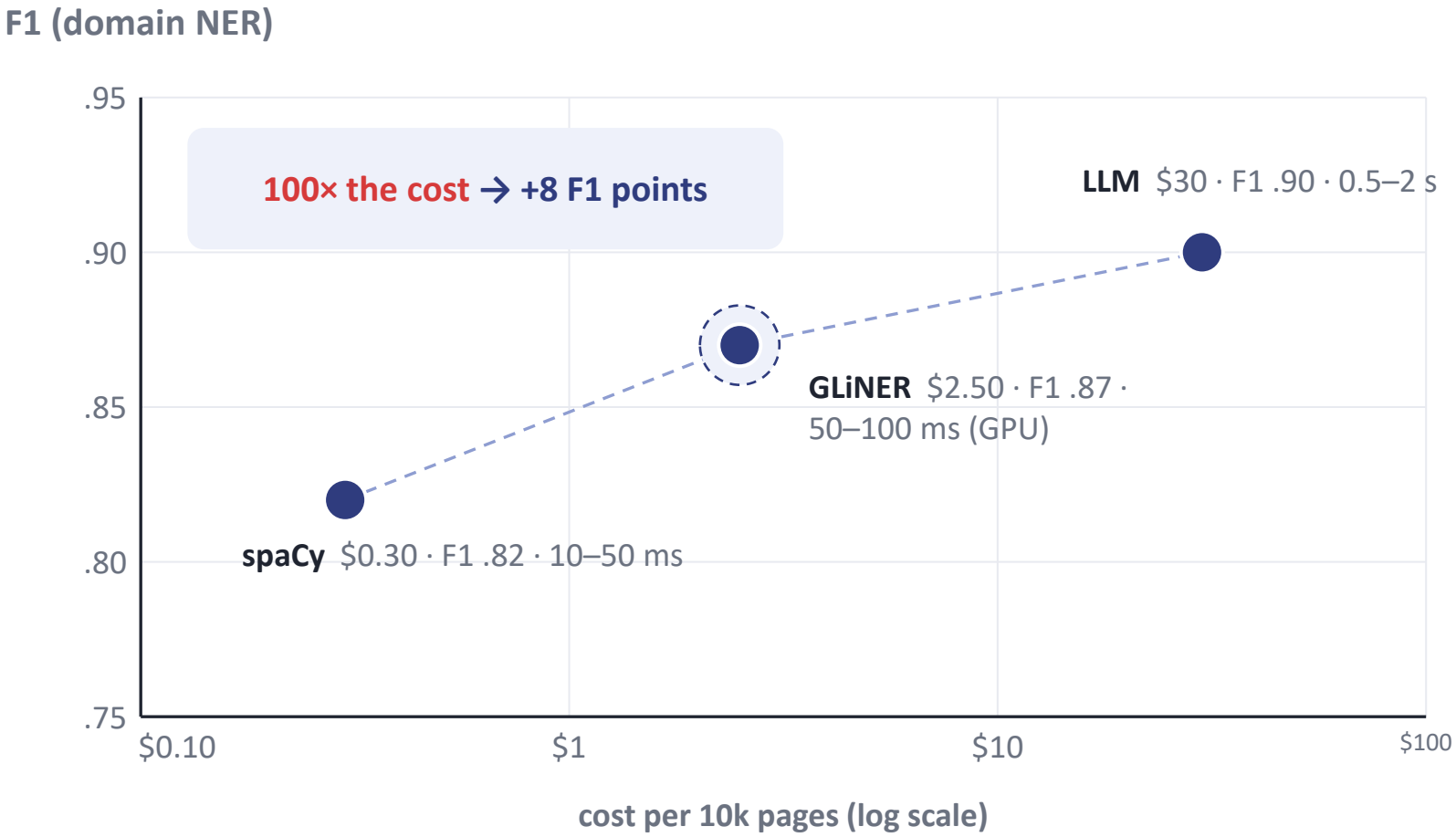
`(ACML)—[OWNS]→(PV-7)`

grounded in =>

DOC-2026-0312 · page 47 · chars [214, 218]
*"...repair scheduled by **ACML** before Q3..."*

No provenance, no triple : an unsourced fact is just a rumor with structure.

Cost / accuracy / latency: real numbers



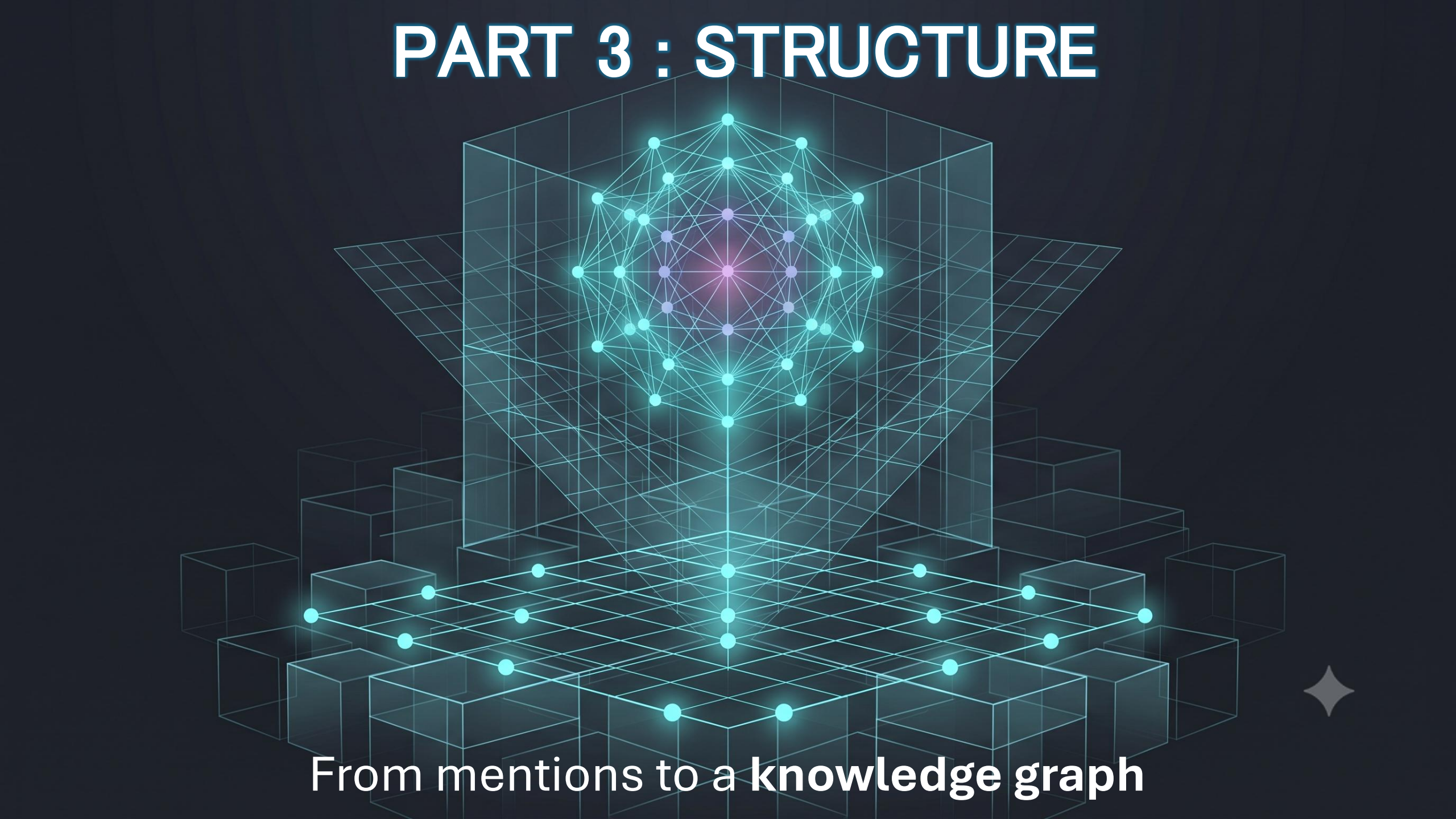
HOW TO READ IT

- The curve flattens: past GLiNER you pay log-scale money for linear gains
- The +8 points live on the hard cases - implicit relations, ambiguity, modality
- So buy them surgically: LLM as a judge pass on disagreements (≈3% of spans)

indicative figures

F1-score: standard metric for evaluating extraction (NER, RE, classif.) = harmonic mean of precision & recall

PART 3 : STRUCTURE



From mentions to a **knowledge graph**

What is a Knowledge Graph

NODE: entity / concept **EDGE: typed, directed relation**

PROPERTY - attribute on a node or an edge

THE ATOMIC FACT - A TRIPLE

`(AIRBUS)—[EMPLOYS]=>(John)`

subject — predicate => object

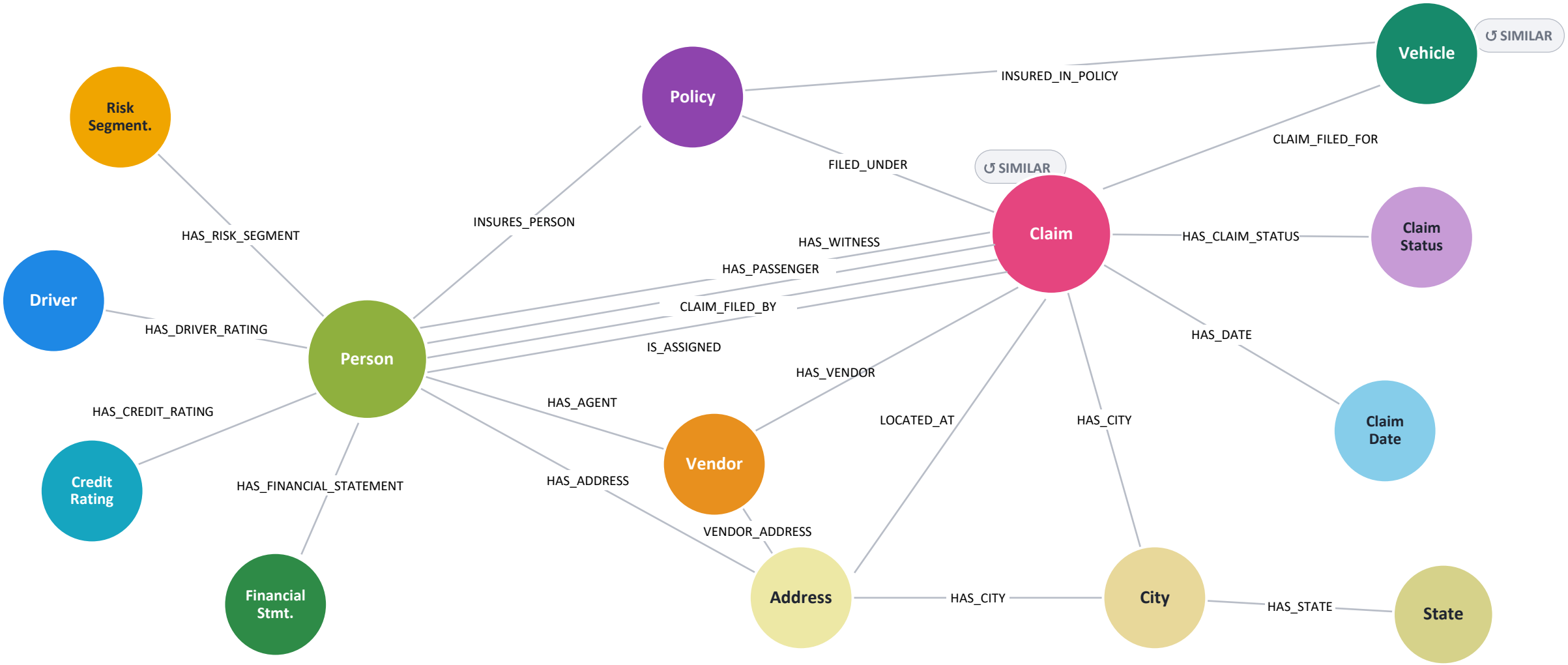
	Relational DB	Doc store	Knowledge graph
Schema	fixed upfront (DDL)	none — implicit	**flexible & typed**
Relationships	JOINs, recomputed at query time	embedded & duplicated	**first-class - traverse directly**
Growth	migration per change	easy but chaotic	**incremental & principled**

Google KG (the search panels) · **Wikidata** (≈115M items) · **DBpedia** (Wikipedia, structured) and enterprise KGs

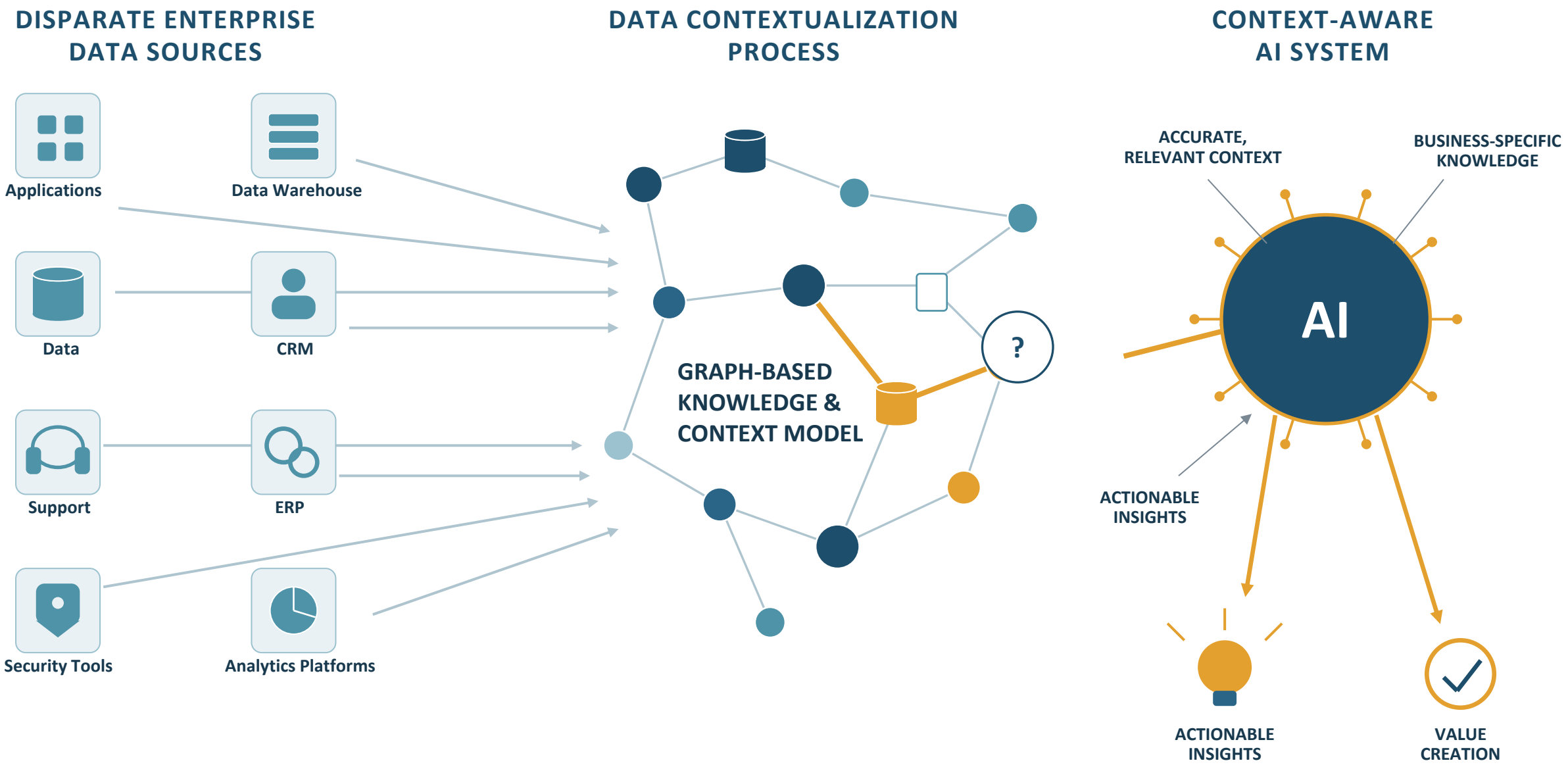
The relationship IS the data - not a JOIN recomputed at every query

An insurance-claim knowledge graph

Two hubs: Person and Claim - tie together policies, vehicles, vendors, ratings and location.



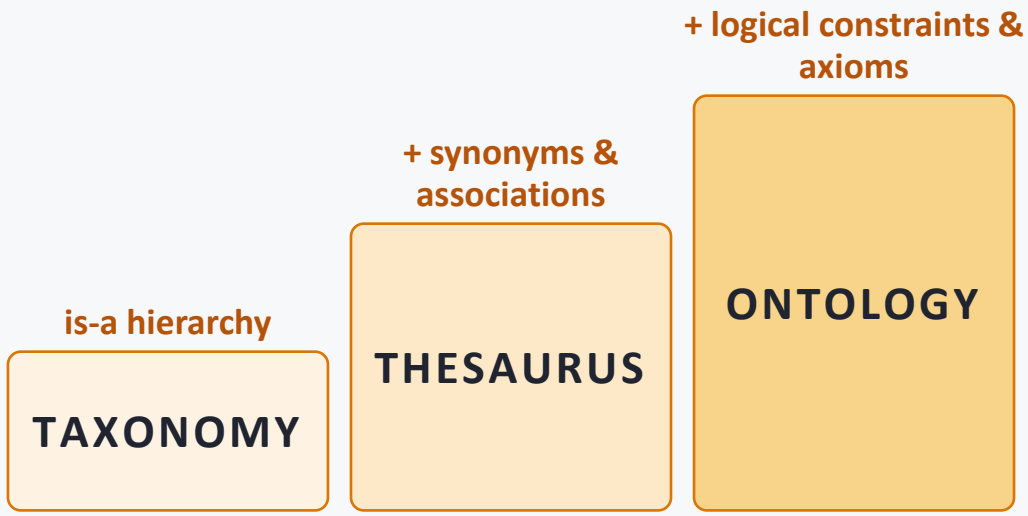
Integrating enterprise data for AI Context



Ontologies: the schema of meaning

Ontology : formal, machine-readable specification of concepts: classes, properties, hierarchies, axioms

THE LADDER - EACH RUNG ADDS MEANING



schema.org / GoodRelations - E-COMMERCE

```
# class + hierarchy
schema:Offer rdfs:subClassOf schema:Intangible .
# property: domain & range
schema:itemOffered schema:domainIncludes
schema:Offer ;
        schema:rangeIncludes schema:Product .
# axiom: every Offer needs a price
schema:Offer rdfs:subClassOf [ owl:someValuesFrom
schema:PriceSpecification ] .
```

an Offer with no price violates the ontology - caught at ingestion

schema.org the web · **GoodRelations** e-commerce · **SNOMED CT** medical · **FIBO** finance · **FOAF** people

Shared vocabulary, validation at ingestion, and inference later

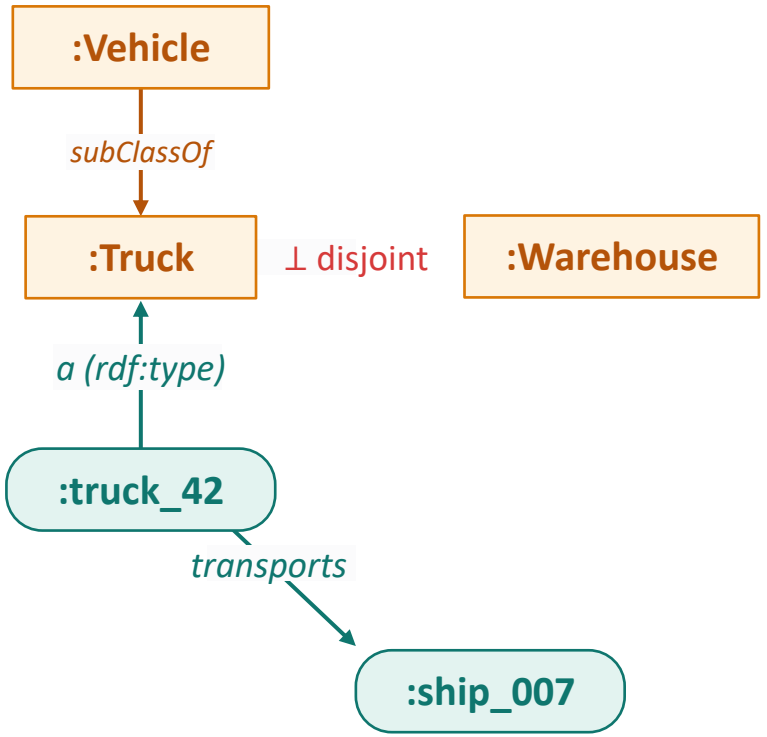
A simple OWL ontology, in Turtle

Classes, a subclass, an object property, one axiom, and an instance

industry.ttl

```
@prefix :     <ex:industry#> .
@prefix owl:  <…owl#> .
@prefix rdfs: <…rdf-schema#> .

:Vehicle  a  owl:Class .                                  CLASS

:Truck    a  owl:Class ;                                  SUBCLASS
          rdfs:subClassOf :Vehicle .

:transports  a  owl:ObjectProperty ;                      PROPERTY
          rdfs:domain :Vehicle ;
          rdfs:range  :Shipment .

:Truck  owl:disjointWith :Warehouse .                     AXIOM

:truck_42  a  :Truck ;                                    INSTANCE
          :transports :ship_007 .
```

THE SAME, AS A GRAPH



Well-known ontologies

Most domains already have standard ontologies

WEB

schema.org

search-engine markup; the web's de-facto vocabulary

METADATA

Dublin Core

15 core fields to describe any resource

PEOPLE

FOAF

persons, accounts & their social links

CONCEPTS

SKOS

thesauri & taxonomies as linked data

PROVENANCE

PROV-O

who, what & when produced data

FINANCE

FIBO

the financial-industry business ontology

LIFE SCIENCES

Gene Ontology · SNOMED CT

gene functions; clinical terminology

HERITAGE & IoT

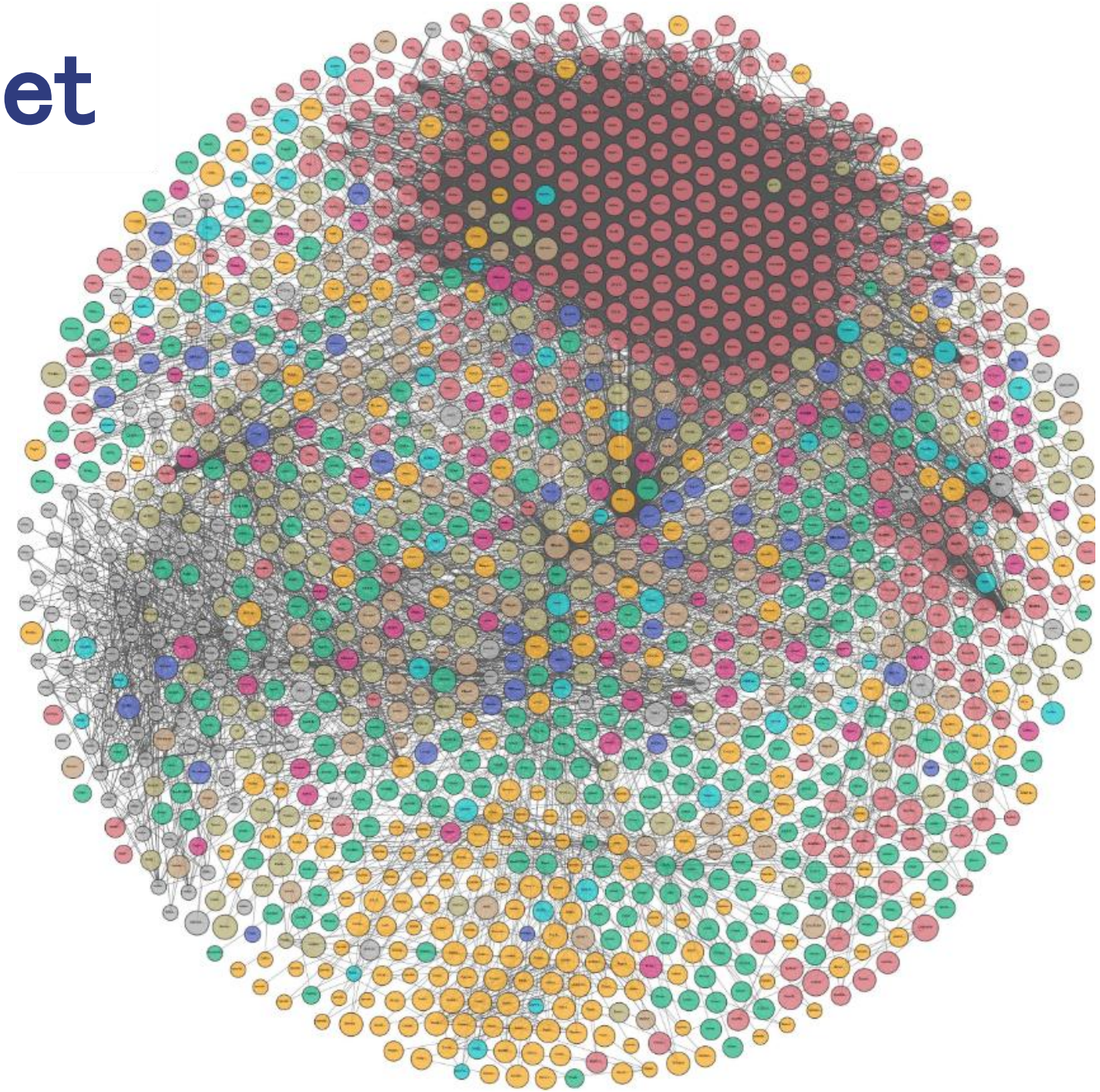
CIDOC-CRM · SOSA/SSN

museum records; sensors & observations

The languages beneath them: **RDF · RDFS · OWL · SHACL** - everything above is built on these.
Upper ontologies (BFO · DOLCE · SUMO) sit at the top; **Wikidata & DBpedia** are the largest open knowledge graphs.

Standard ontologies may already cover your domain.

lod-cloud.net



Cross Domain
Geography
Government
Life Sciences
Linguistics
Media
Publications
Social Networking
User Generated

DBpedia: Wikipedia as a Knowledge Graph

Structured facts pulled from Wikipedia into RDF - the anchor of the open web of data.

FROM WIKIPEDIA TO RDF

Wikipedia infobox — "Strasbourg"

```
Country:   France
Region:    Grand Est
Population:  291,313
```

extract · map to the DBpedia ontology (dbo:)

```
dbr:Strasbourg a dbo:City .
dbr:Strasbourg dbo:country dbr:France .
dbr:Strasbourg dbo:region dbr:Grand_Est
  .
dbr:Strasbourg dbo:populationTotal
  291313 .
```

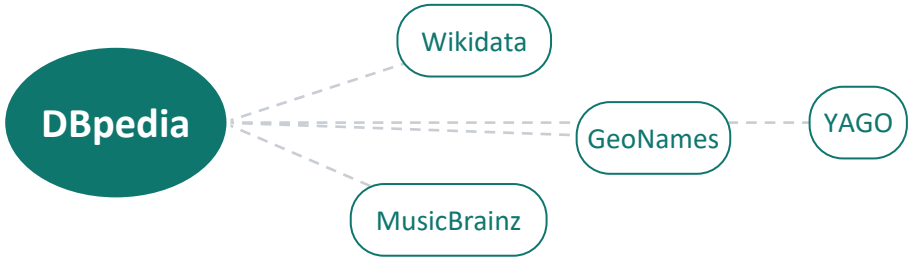
768 classes
3,000 properties
the cross-domain ontology

220M entities
1.45B triples
latest DBpedia release

100+
language editions
of Wikipedia extracted

since 2007
open & free
a founding LOD hub

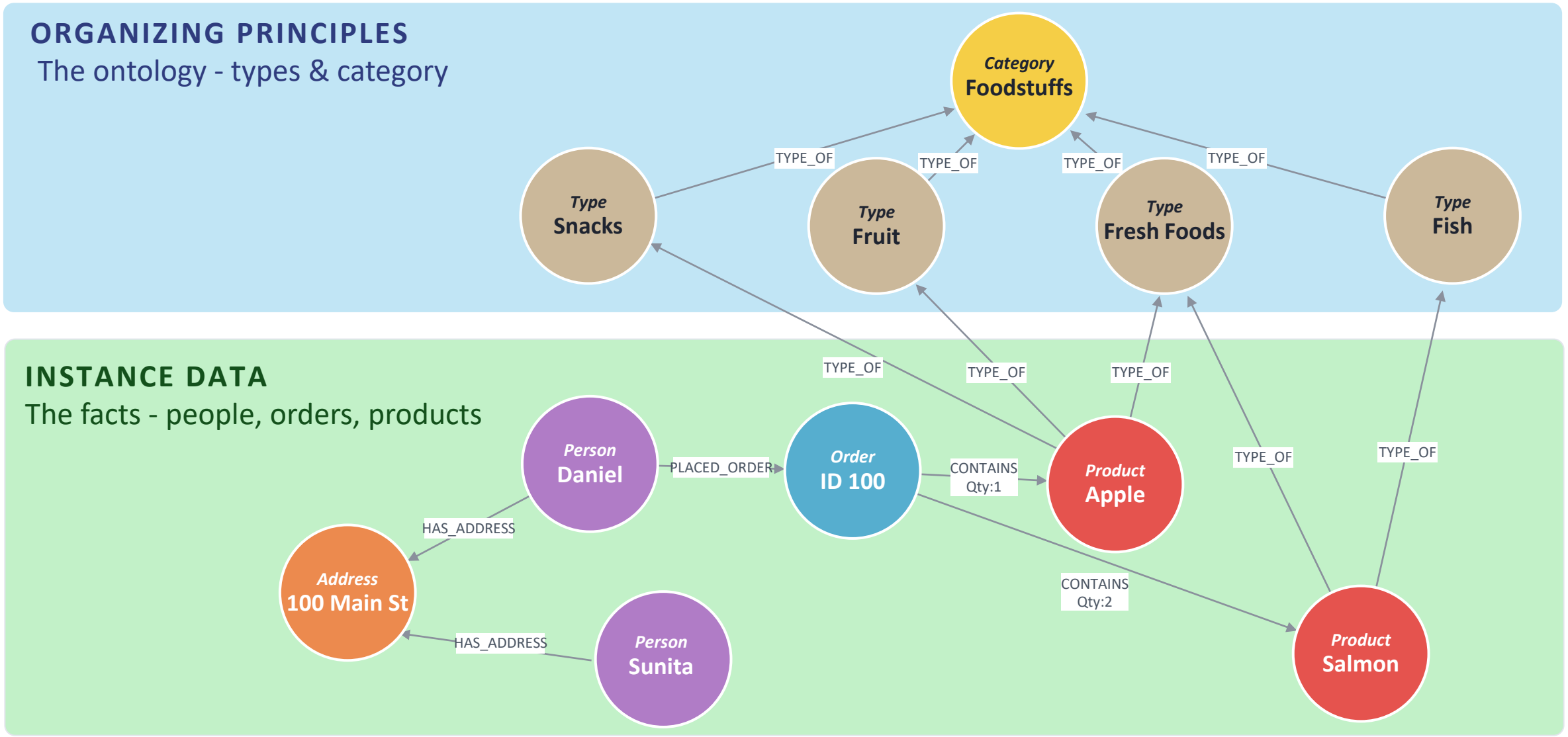
THE HUB OF LINKED OPEN DATA



DBpedia turns Wikipedia into a queryable graph - one SPARQL endpoint over the world's encyclopedia.

Schema and data in one graph

The ontology sits above the instances it describes - **TYPE_OF** is the bridge.



Concept granularity

Same fact: “Maître Weber signs a deed for the Muller sale”

TOO COARSE

Who represented whom? Unanswerable - the roles are gone.

JUST RIGHT

Notary is one level below Person - specific enough to query, general enough to reuse.

TOO FINE

One instance ever. Extraction breaks; nothing else links here.

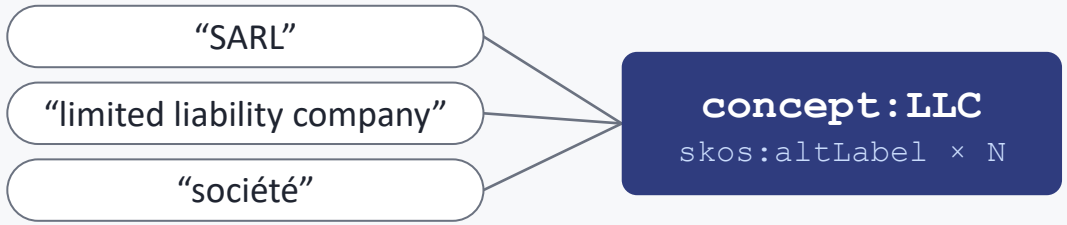
HEURISTICS TO FIND THE MIDDLE

1. **Start from competency questions** - let the questions you must answer define the classes
2. **One level below your queries** - you ask about “notaries” => model Notary, not Person, not SellerRep
3. **Collapse rare relation types into broader ones**- too few instances of a relation? collapse it into a broader one

Granularity is dictated by the questions the graph must answer.

Synonyms & hypernyms

SYNONYMS: N SURFACE FORMS, 1 CONCEPT



HYPERNYMS: QUERY AT ANY LEVEL

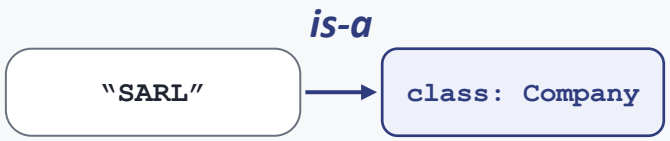
Notaire *is-a* **LegalProfessional** *is-a* **Person**

“list all persons in the deed” also returns the notary

Alignment, entity resolution & linking

SEMANTIC ALIGNMENT
term => concept / class

"What KIND of thing is this?"



needs an ontology

- maps to a TYPE, not an individual
- "B.V." => class Organization
- the umbrella term - also covers schema-to-schema alignment

what kind?

ENTITY RESOLUTION
mention <=> mention

"Are these the SAME one?"

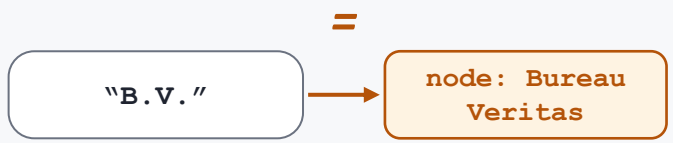
needs NO directory - builds one

- dedupes mentions into clusters
- blocking + matching (n^2 without blocking)
- works even for entities nobody has catalogued yet

same as each other?

ENTITY LINKING
mention => known node

"WHICH known one is this?"



needs an existing referential

- disambiguates to an INSTANCE
- look-up in a directory (your KG, Wikidata)
- fails if the entity isn't in the directory

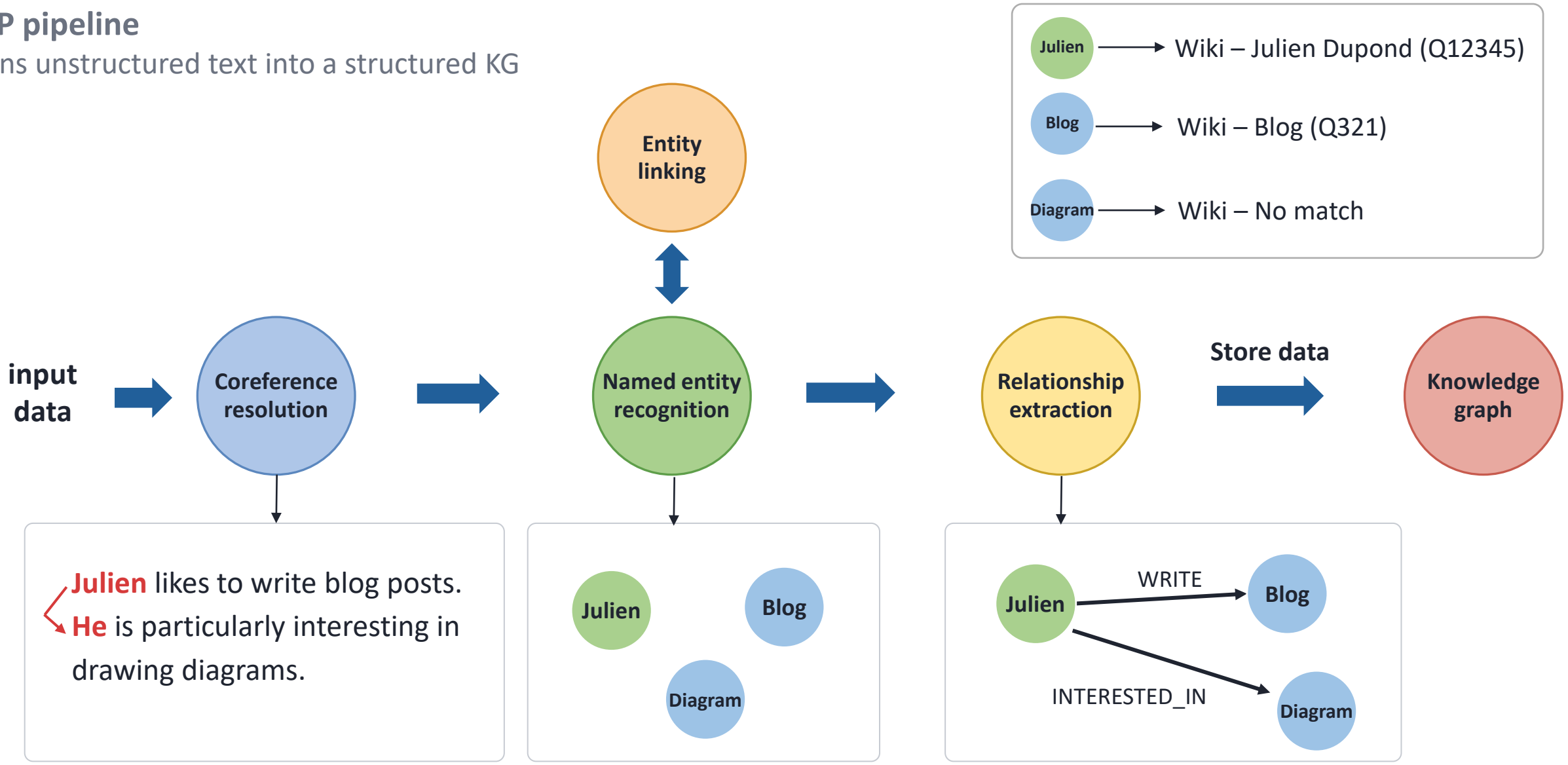
which known one?

Resolution builds the directory, linking looks things up in it - alignment says what kind they are.

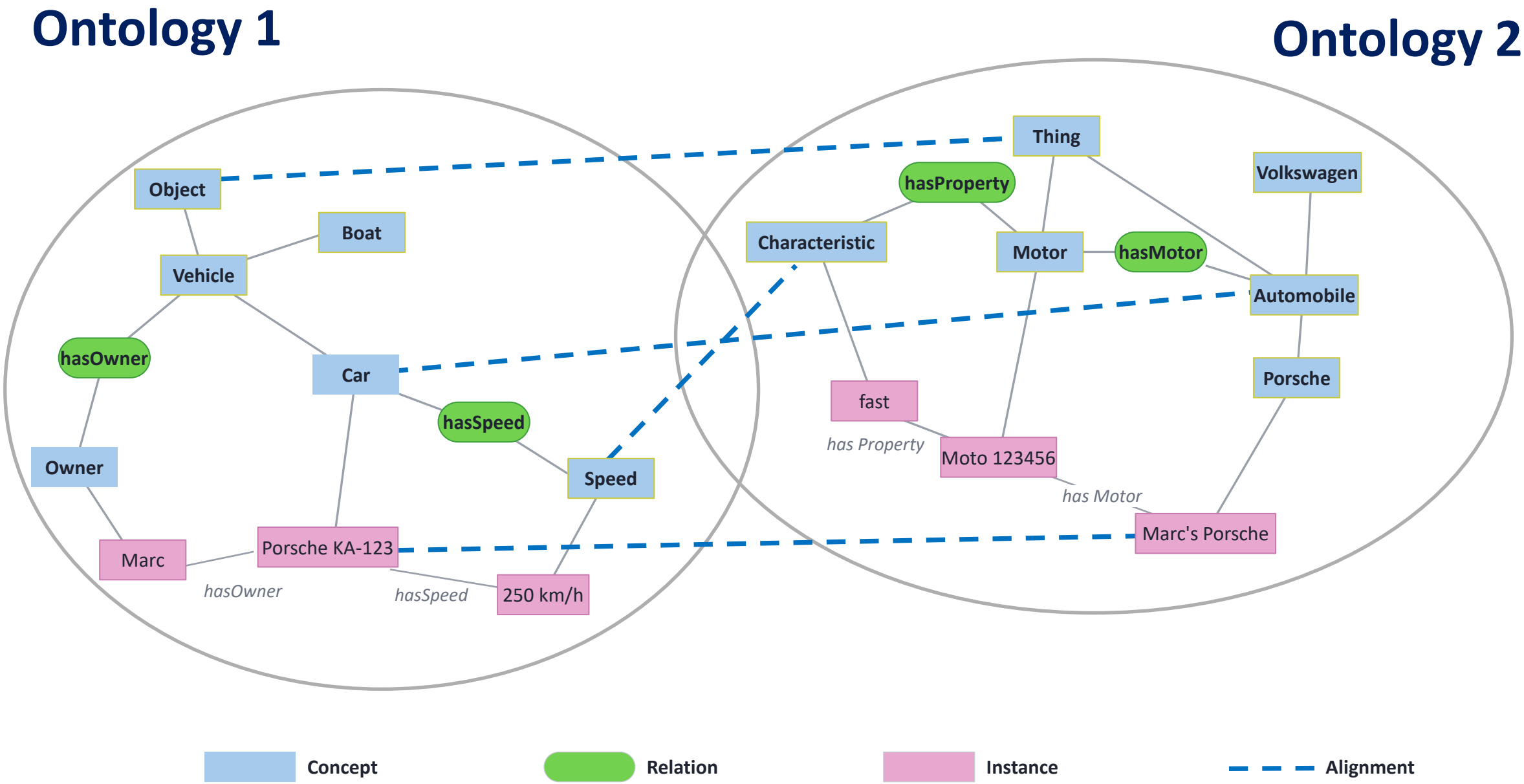
From text to knowledge graph

NLP pipeline

Turns unstructured text into a structured KG

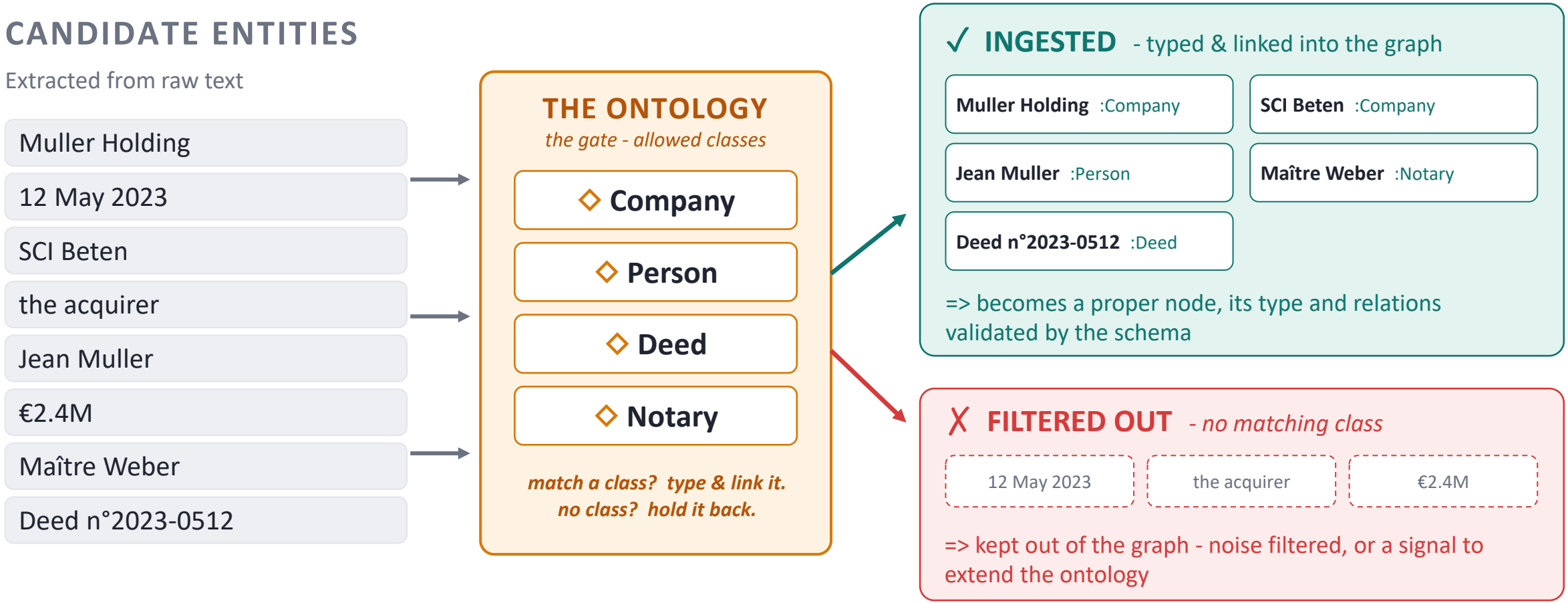


Ontology Alignment



Ontology as an ingestion gate

At ingestion, the ontology decides what becomes a typed node and what gets filtered out.



The ontology is the gatekeeper - only what fits the schema becomes typed knowledge.

Automated ontology generation

CORPUS TO SCHEMA

▲ *humans stay in the loop*
the LLM proposes, people dispose

THE CATCH - GREAT DRAFTER, BAD LOGICIAN

✓ surprisingly good at **drafting** a first ontology from raw text

✗ terrible at **consistency**: duplicate classes, cycles in is-a, contradictory ranges

=> **fix: validate every draft with a reasoner / SHACL constraints**

A LIVING ARTIFACT - VERSION IT

```
propose →
validate →
curate →
release vN ↻
```

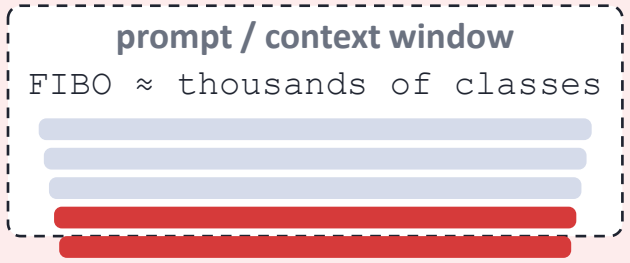
diff it, review it, roll it back - like code

Let the LLM draft the ontology - then never trust it without a reasoner.

RAG over the ontology itself

Ontology is too big for the prompt? Retrieve it, like any other knowledge

PROBLEM



won't fit and dilutes attention

stuffing the whole schema =>
the model ignores most of it and mislabels

SOLUTION: TWO-STAGE PROMPTING

offline, once: embed every class & property *definition* => a vector index of the schema

STAGE 1 - WHICH CONCEPTS APPLY?

Chunk *"…acquired all shares…"*

↓ embed + top-k over schema

Retrieved sub-schema:
Company · Ownership · Acquisition

STAGE 2 - EXTRACT WITH ONLY THOSE

prompt = chunk + **3 classes (not 1000)**
↓
`(Muller)—ACQUIRED→(SCI Beten)`

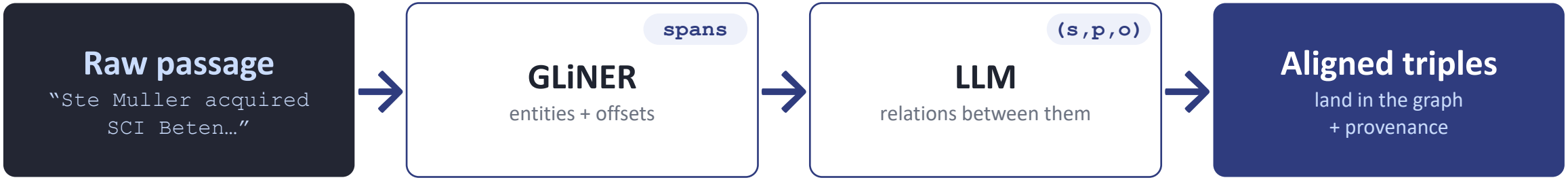
Same trick as document RAG - **retrieve the relevant slice, ignore the rest**

Fewer choices => fewer mistakes: narrowing the label set is the single biggest lever on extraction consistency at scale

Don't prompt the whole ontology - retrieve the part that matters, per chunk.

Checkpoint – Text to graph

END TO END

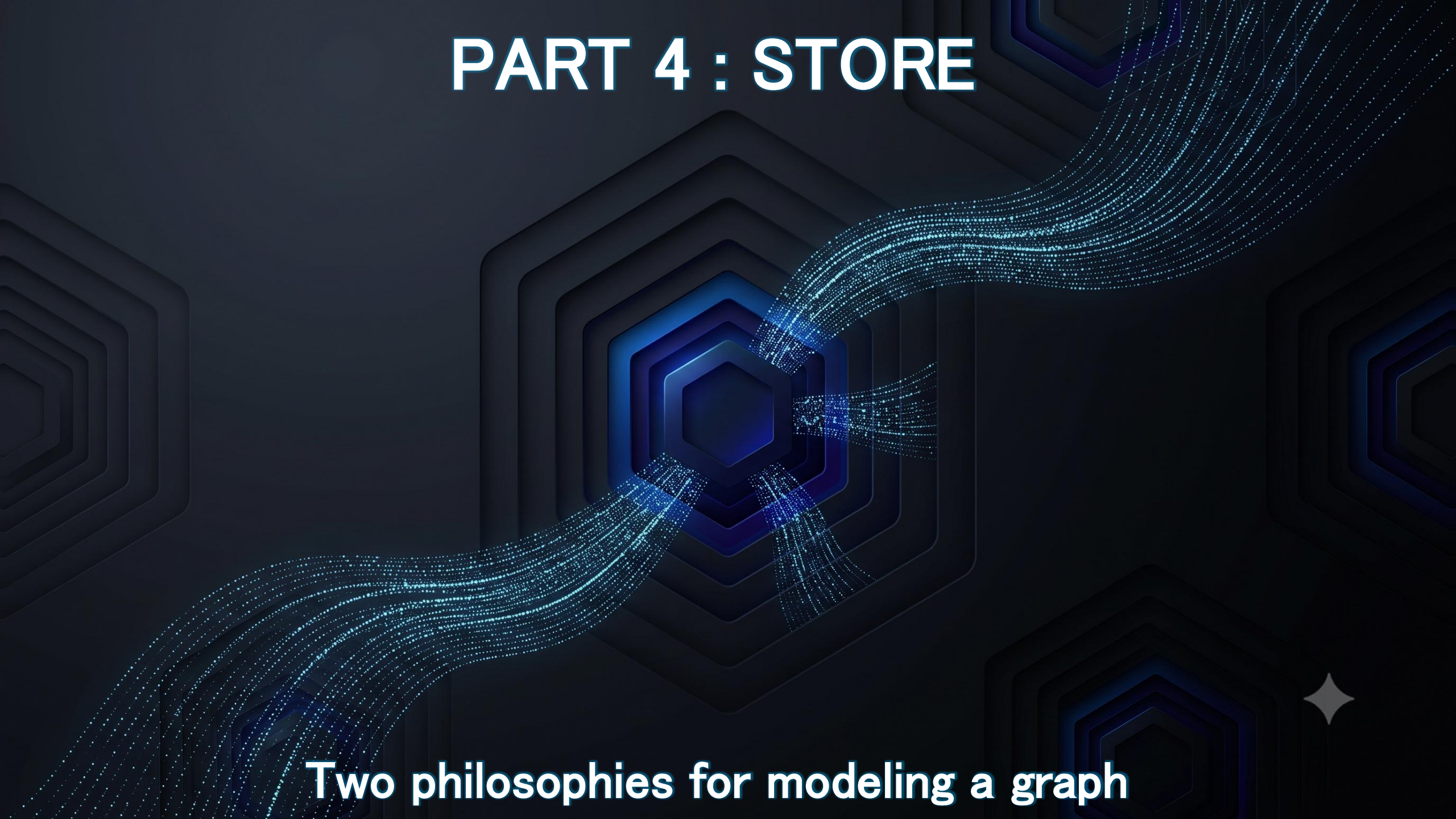


✓ **with alignment:** two names => **one node**

✗ **without it:** 2 nodes - the graph silently splits the company

Every triple lands with a source span - and two names for one firm land on one node.

PART 4 : STORE



Two philosophies for modeling a graph

RDF in a Nutshell

Resource Description Framework: the W3C's data model for the Semantic Web

WHAT IT IS

A way to state facts as triples :

millions of triples = a directed graph. Every resource is a URI.

WHERE IT COMES FROM

1997	first W3C draft - IBM, MS, Netscape, Nokia, Reuters
1999	W3C Reco - metadata for the Web (Tim Berners-Lee)
2004 / 14	RDF 1.0, then 1.1 - SPARQL, RDFS, OWL, SHACL

born to describe web resources - foundation of the Semantic Web

✓ Interoperability
Shared URIs let independent datasets merge
Wikidata, DBpedia

✓ Machine-readable meaning
The schema (RDFS/OWL) travels with the data

✓ Symbolic inference
Reasoners derive new triples from axioms, for free

RDF turns data into a web of facts - global, mergeable, and reasoned over.


RDF TRIPLE STORE:
SEMANTIC, RIGOROUS, STANDARDS
LPG DATABASE:
PERFORMANCE, FLEXIBLE, PRAGMATIC
RDF Triple Store
vs.
LPG Database:
THE GRAPH MODEL CONFLICT
TRIPLE STORE
SPARQL
W3C STANDARDS
IRIs
<http://example.org/resource>
S
P
O
GRAPH TRAVERSAL
LABELED PROPERTY GRAPH
CYPHER
KEY-VALUE PAIRS
REAL-TIME ANALYTICS
KNOWLEDGE GRAPH MODELS

RDF Triple Stores

Everything is a triple. The W3C standards stack, and reasoning for free.

THE MODEL - TRIPLES ALL THE WAY DOWN


```
:Muller      :acquired      :SCI_Beten .
```

URIs everywhere · schema = OWL / RDFS ontology

QUERY - SPARQL · W3C STANDARDS STACK

```
SELECT ?c WHERE {
  :SCI_Beten :ownedBy ?c }
```

RDF · RDFS · OWL · SHACL · SPARQL

SUPERPOWERS

- ✓ **Formal semantics:** meaning is machine-defined, not convention
- ✓ **Reasoners - inference for free:** new facts derived from axioms
- ✓ **Federation & interoperability:** join across datasets by shared URIs

WEAKNESSES

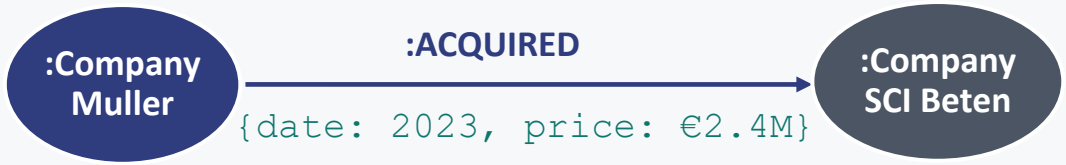
- ✗ **Developer ergonomics:** verbose; steeper learning curve
- ✗ **Property-heavy data is awkward:** needs reification / RDF-star
- ✗ **Ops maturity:** smaller tooling & talent pool

Triple Stores: GraphDB · Apache Jena / Fuseki · Virtuoso · Blazegraph · Amazon Neptune · Stardog

LPG Databases

Labeled Property Graphs - nodes and relationships as first-class citizens, with properties.

THE MODEL - PROPERTIES ON EVERYTHING



Properties live on BOTH nodes and relationships · no mandatory schema

QUERY - CYPHER (NOW ISO GQL)

```
MATCH (c:Company)-[:ACQUIRED]->
      (:Company {name:'SCI Beten'})
RETURN c
```

Neo4j · Memgraph · FalkorDB · Neptune

SUPERPOWERS

- ✓ **Developer experience:** intuitive model, gentle on-ramp
- ✓ **Traversal & path algorithms:** GDS - shortest path, PageRank, communities
- ✓ **Native vector indexes:** graph + embeddings in one store

WEAKNESSES

- ✗ **No built-in formal semantics**: the engine doesn't know what a label means
- ✗ **No native reasoning:** inference is app code, not the database
- ✗ **Ontology = convention:** a naming habit, not an enforced contract

LPG is what you ship: fast, pragmatic, developer-first - but the ontology is on you.

Two philosophies, two use cases

RDF - knowledge interchange

When symbolic inference & standards matter

- Public / open data (Wikidata, gov)
- Regulated & compliance domains
- Cross-org data federation
- Formal semantics + reasoning
- Semantic GraphRAG - symbolic inference

LPG - knowledge applications

When speed, vectors & iteration matter

- Product features on a graph
- GraphRAG : graph + vectors
- Fast traversal & path analytics
- Rapid schema iteration

THE PRAGMATIC PATTERN - HAVE BOTH

MODEL
with an ontology

→

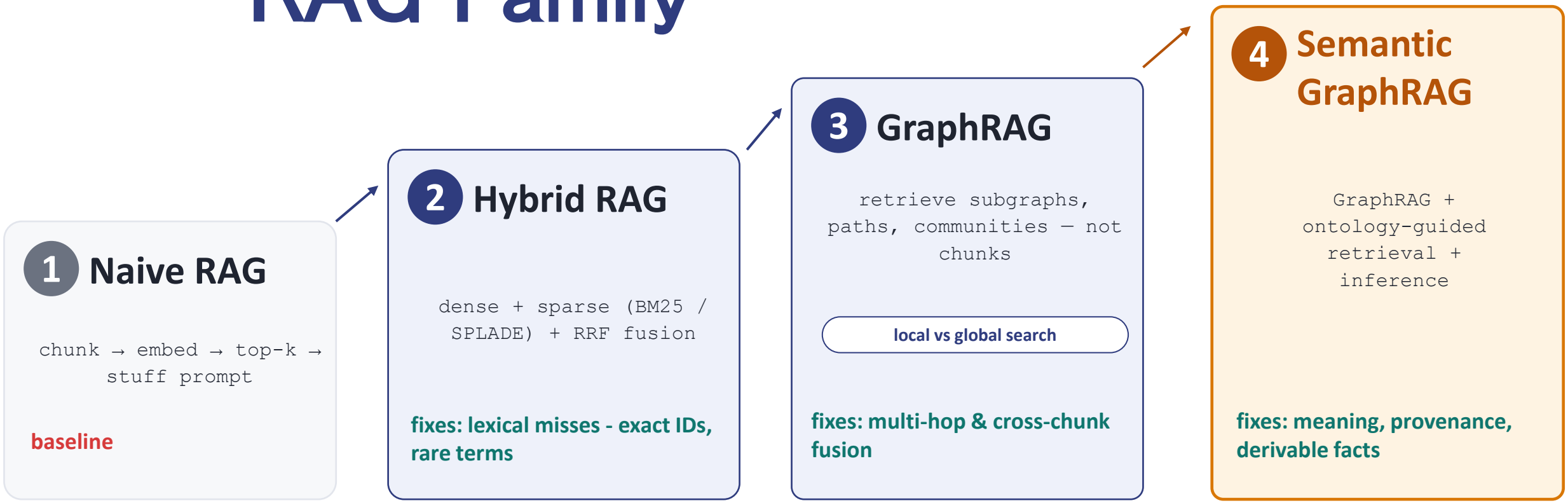
OPERATE
on LPG

OWL classes → node labels
properties → relationship types
SHACL-style checks → in the ingestion pipeline

PART 5 : QUERY

Retrieval is where representation pays off

RAG Family



dense - vector embedding search, which matches semantic meaning even when no words are shared.
sparse - lexical search over token-indexed sparse vectors, which matches exact words.
BM25 - the classic lexical scoring algorithm, weighting term frequency against corpus rarity (an improved TF-IDF).
SPLADE - neural sparse: a model learns to weight and expand a document's tokens while keeping lexical interpretability.
RRF (Reciprocal Rank Fusion) - merges rankings from several engines via `score = Σ 1/(k + rank)`, with no need to normalize heterogeneous scores.
lexical misses - cases where dense alone fails because semantic similarity dilutes the precise token being searched for.

Vector DBs: What Actually Differs

WHAT TO COMPARE ON

Index type HNSW · IVF · DiskANN

Filtering strategy pre / post / hybrid

Quantization scalar · binary — memory vs recall

Hybrid search dense + sparse in one query

Metadata capabilities rich filters, payload types

Ops model self-host vs managed

highlighted = where products actually diverge

QUICK LANDSCAPE

★ **pgvector** - already in Postgres

- **Qdrant** - filtering + quantization
- **Weaviate** - hybrid search built-in
- **Milvus** - built for scale
- **Pinecone** - fully managed

★ **Neo4j vector index** - graph + vectors, one engine

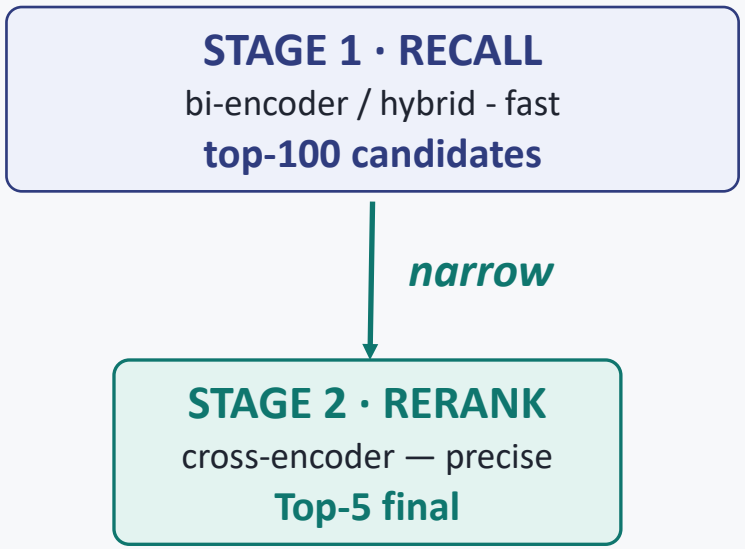
★ = covers most GraphRAG cases with no new database

The index math is commoditized. Filtering + hybrid + ops are the real differentiators - not HNSW vs IVF.

Rerankers

Retrieve wide and cheap, then re-score the shortlist precisely.

TWO-STAGE RETRIEVAL



cheap: rerank 100, not the whole corpus

RERANKER TYPES

- **Cross-encoders** BGE-reranker · mxbai
- **Late interaction** ColBERT - token-level
- **LLM-as-reranker** prompt to score/order
- **API rerankers** Cohere · Voyage

WHY CROSS-ENCODERS WIN ON PRECISION

bi-encoder: encode Query and Doc **separately** => compare vectors

cross-encoder: Query and Doc in **one pass** - they attend to each other

more accurate, too slow for the whole corpus - perfect on a top-100 shortlist

A small reranker often beats a bigger embedding model - add it before you swap anything.

Embeddings: finding the entry points

What gets embedded in a graph, and how it's scored for similarity against your question.

① WHAT GETS EMBEDDED

not raw chunks alone - graph elements:

Node + its description
"Muller Holding - a Company…"

Relationship
"acquired · signed before…"

Source text chunk
the passage it came from

Community summary
a whole subgraph, in words

② EMBEDDING MODEL

one model, one shared vector space

USER QUESTION
"Who controls SCI Beten?"

same model embeds it too

③ COSINE SIMILARITY

angle between the question vector and each stored vector — 1 = same meaning, 0 = unrelated

SCI Beten	0.91
Muller Holding	0.86
Jean Muller	0.70
Boat	0.09

=> top-k = the ENTRY POINTS

Similarity picks where to start - then the graph takes over from those entry nodes.

Hybrid queries on Neo4j

Deterministic meets semantic - embeddings to find, the graph to connect.

THE PATTERN

① VECTOR SEARCH - find entry points

Semantic: “companies like this one” => top-k nodes

② GRAPH TRAVERSAL - collect facts

Deterministic: follow OWNS / SIGNED edges, 1–3 hops

⇄ INVERSE PATTERN

Known SIREN => traverse =>
semantic expansion on neighbors

ONE QUERY, BOTH WORLDS

cypher

```
CALL db.index.vector.queryNodes
  ('entity_emb', 5, $qvec)
  YIELD node, score
MATCH (node)-[:OWNS|SIGNED*1..3]-(related)
RETURN node, related, score
```

<= ① vector search

<= ② traversal

This is the core mechanic of GraphRAG: embeddings for finding, the graph for connecting.

Retriever types: entry point & reach

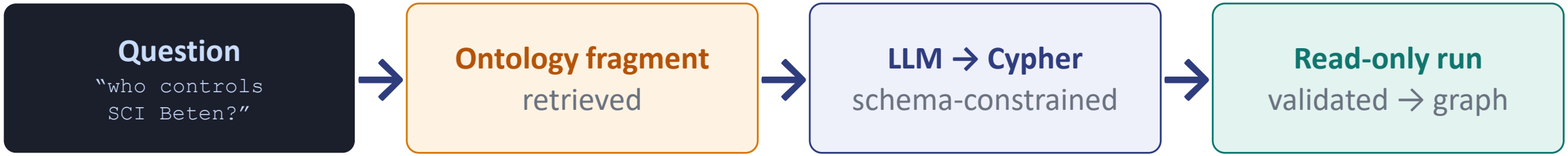
How each retriever seeds, how far it expands into the graph, and when to reach for it.

RETRIEVER	ENTRY POINT	EXPANDS IN GRAPH	BEST FOR
Vector chunk RAG	top-k similar chunks	**none - flat text**	fast text QA
Lexical (BM25)	keyword match	**none - flat text**	exact terms & names
Summaries	similar summaries	**none - condensed**	quick overviews
Triplet	similar embedded facts	**seeds only, no traversal**	precise facts, cheap
Graph completion	vector-seeded nodes/edges	**resolve=> 1-hop neighbourhood**	relationship-aware answers
Hybrid	BM25 + vector + entity	**entity edges + related facts**	precision + recall + structure
Iterative (multi-hop)	vector seeds	**follow-up rounds, hop by hop**	deep chains A→B→C→D
NL→Cypher / Cypher	schema-driven query	**query-defined traversal**	exact structured results

GRAPH DEPTH: no graph (flat text) seeds only seed + expand structured query

Text–to–Cypher & retrieval agents

TEXT-TO-CYPHER - QUESTION BECOMES QUERY



Retrieved ontology fragment constrains generation

GUARDRAILS - GENERATED CODE IS UNTRUSTED

- ✓ **Schema-aware generation** - only real labels & types
- ✓ **Query validation** - parse & check before running
- ✓ **Read-only execution** - no writes, ever
- ✓ **Fallback to templates** - known-good query when unsure

AGENTIC RETRIEVAL - A LOOP, NOT ONE SHOT

LLM picks the retrieval tool, reads the result, decides the next move:

vector | **traversal** | **Cypher** | **community summary**

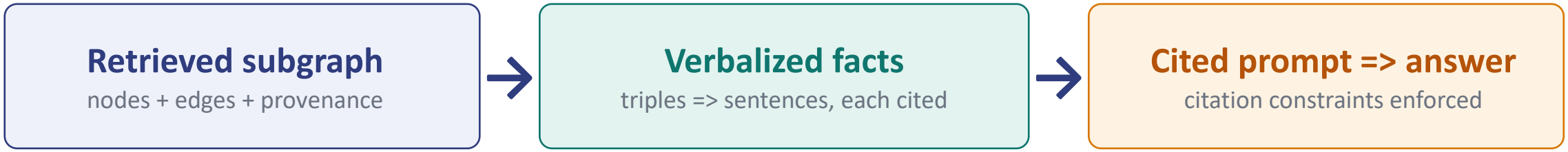
↻ **iterate until the answer is grounded - retrieval as reasoning**

Retrieval stops being a single lookup - it becomes a loop the model drives.

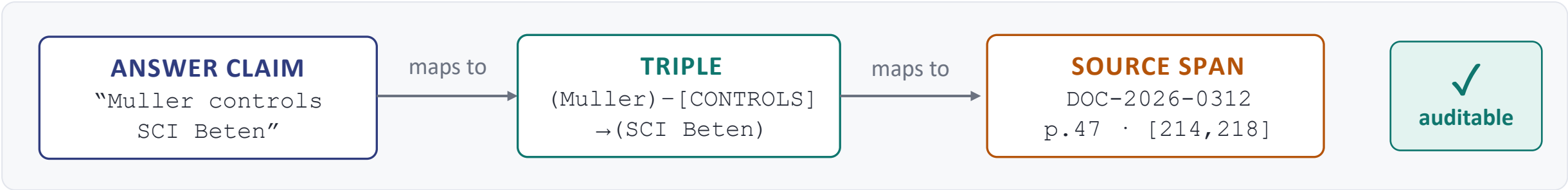
Answer assembly & grounding

From subgraph to a cited answer - auditability by construction.

THE ASSEMBLY FLOW



EVERY CLAIM TRACES BACK - THE AUDIT CHAIN



This is the answer to hallucinated relationships: no source span, no claim

Checkpoint – Naive RAG vs GraphRAG

QUESTION "Who ultimately controls SCI Beten?" - *A 3-hop ownership chain across three documents*

NAIVE RAG ✗ wrong

chunks retrieved by similarity - the chain is split across them

> *"**Muller Holding** controls SCI Beten."*

- ✗ **stops at hop 1** - misses that Jean Muller controls the holding
- ✗ **no sources to check**

GRAPHRAG ✓ right

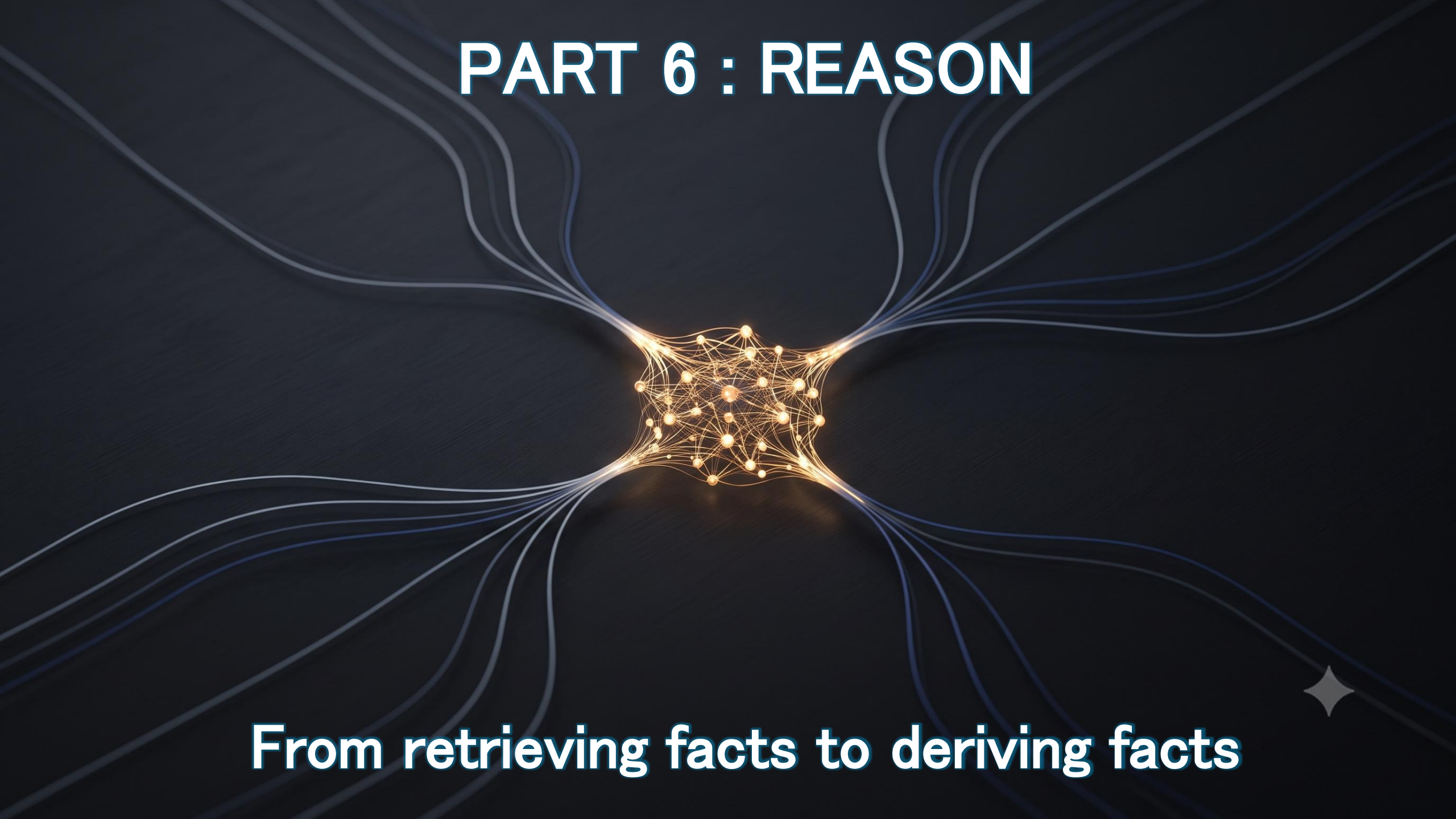
`SCI Beten ←owned– Muller Holding ←ctrl– Jean Muller`

> *"Ultimately **Jean Muller**, via Muller Holding."*

- ✓ **traverses all 3 hops** — the graph has the full chain
- ✓ **every hop cited:** `3 deeds · pages + offsets`
- ✓ **provenance chain intact** — auditable end to end

Same question, same corpus - the multi-hop answer only exists in the graph.

PART 6 : REASON



From retrieving facts to deriving facts

Symbolic AI : axioms & logic

THE LANGUAGE

Propositional

"it rains" = one true/false atom - no structure

First-order logic

adds structure you can reason over:

predicates
`owns(x, y)`

variables
`x, y, z - stand for anything`

quantifiers
`∀ for all · ∃ there exists`

AXIOMS FROM OUR DOMAIN - RULES THE REASONER APPLIES

① Transitivity of control

$$\forall x,y,z:\ \ owns(x,y) \land owns(y,z) \rightarrow controls(x,z)$$

Muller owns Holding, Holding owns Beten ⇒ Muller controls Beten

② Inverse role

$$\forall x,y:\ \ signedBy(x,y) \rightarrow party(y,x)$$

③ Disjointness - catches extraction errors

$$Person \sqcap Company = \bot$$ one mention can't be both ⇒ flags the merge

Inference = mechanically deriving new true statements from axioms + facts. **Sound · explainable · deterministic.**

No embeddings, no probabilities - just rules that turn facts into more facts, provably.

More axioms, real values

1 **Part-of transitivity**

$\forall x,y,z:\ \ partOf(x,y) \land partOf(y,z) \rightarrow partOf(x,z)$

weld **W-105** ▸ vessel **PV-7** ▸ **Hall B** ⇒ **W-105 partOf Hall B**

2 **Class inheritance (subsumption)**

$\forall x:\ \ isA(x,C) \land subClassOf(C,D) \rightarrow isA(x,D)$

PV-7 isA **PressureVessel** ⊑ **RegulatedEquipment** ⇒ **PV-7 is regulated**

3 **Spatial transitivity**

$\forall x,y,z:\ \ in(x,y) \land in(y,z) \rightarrow in(x,z)$

Plant **in Strasbourg** ▸ **in Grand Est** ⇒ **Plant in Grand Est**

4 **Symmetry**

$\forall x,y:\ \ joinedTo(x,y) \rightarrow joinedTo(y,x)$

segment **S1** joinedTo **S2** ⇒ **S2 joinedTo S1**

5 **Composition → obligation**

$\forall x,w:\ \ hasPart(x,w) \land failed(w) \rightarrow needsRepair(x)$

PV-7 hasPart **W-105**, weld **FAILED** ⇒ **PV-7 needsRepair**

Symbolic inference in practice

How reasoning actually runs - in both storage worlds - and where it breaks.

RDF WORLD - NATIVE REASONERS

- **OWL reasoners:** HermiT · ELK - apply axioms
- **Materialize vs query-time:** precompute all facts, or infer on ask
- **SHACL:** validate the graph against shapes

LPG WORLD - RULES YOU WRITE

inference as Cypher rules, run at ingestion:

```
if OWNS chain > 50%
→ CREATE (:CONTROLS) edge
```

- or bolt on a dedicated rules engine

KILLER USE CASES

⚠ **Consistency checking**
find contradictions in extracted data

➕ **Completing implicit facts**
indirect ownership, derived control

§ **Regulatory rules as axioms**
compliance encoded, auto-applied

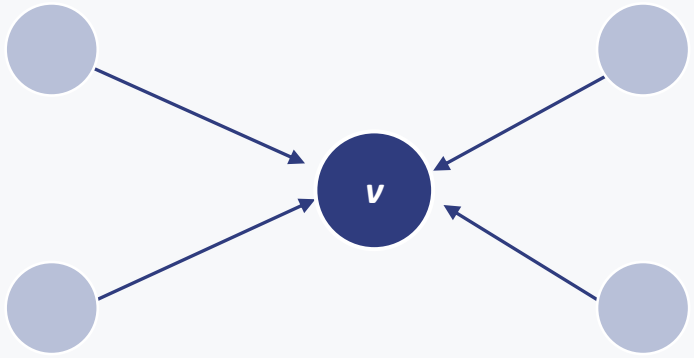
LIMITS

Open-world assumption (absence ≠ false) · **Brittle to noisy extraction** (garbage in → garbage inferred) · **Cost of expressive logics** (rich OWL can blow up)

GNNs: learning on graphs

Graph Neural Network: pattern-learning where logic won't reach

WHAT THEY ARE: MESSAGE PASSING



Each node's vector = f(its neighbors' vectors)

Stack layers => node / edge / graph-level predictions

WHERE THEY FIT IN THIS PIPELINE

Link prediction

Suggest missing edges - for HUMAN review

Entity resolution scoring

How likely are two nodes the same?

Node classification

Infer a type from graph position

KG embeddings

TransE => GNN-based vectors of nodes

GNN: learned **plausibility**

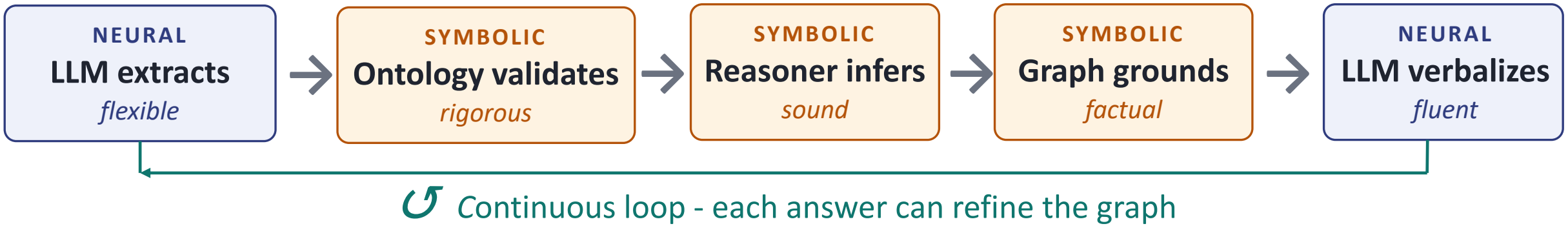
Symbolic: proven **certainty**

Complementary, not competing - GNNs guess what to check, logic proves it.

Neo4j GDS: GraphSAGE · node2vec

Neuro-symbolic AI

Neural for language, symbolic for truth



EACH COVERS THE OTHER'S WEAKNESS

LLMs handle ambiguity & language

Symbols handle consistency & multi-hop truth

Neither is enough alone - together they close most failures

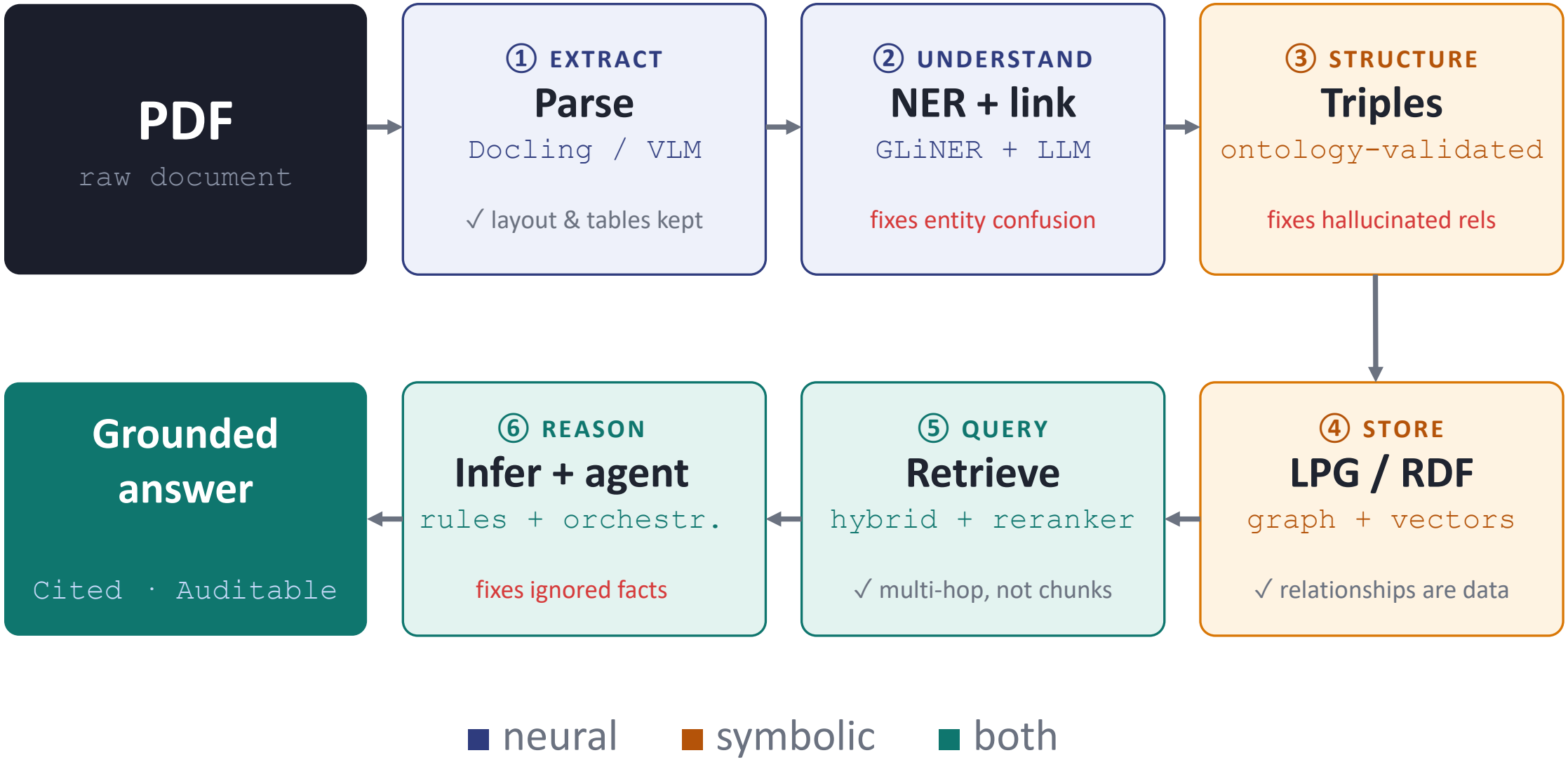
THE ORCHESTRATOR - RETRIEVER AGENTS

The agent chooses, on the fly:

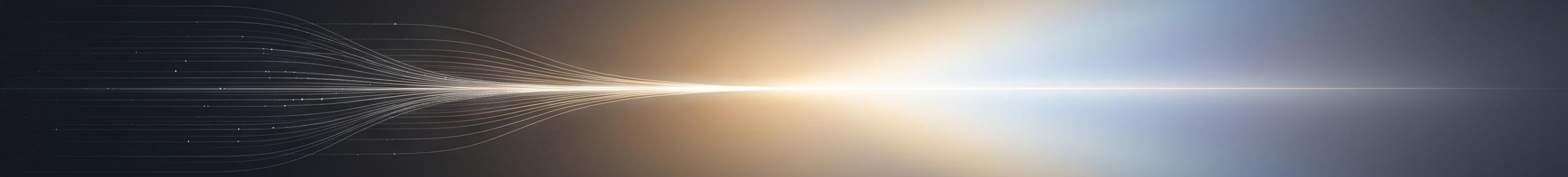
- vector search
- traversal
- Cypher gen
- inference

This is Semantic GraphRAG

Full architecture recap



PART 7 : CONCLUSION



What to Take Home

1. **Representation beats model size.**
   Structure - not a bigger model - is the fix for hallucinated relationships.

2. **Right tool per layer.**
   GLiNER ≠ spaCy ≠ LLM - cost / accuracy / latency is a choice you make **at every stage**.

3. **Start simple, add structure where questions demand it.**
   `naive => hybrid => GraphRAG => semantic`

4. **An ontology is an investment that compounds.**
   Extraction quality, query generation, validation, inference — all improve at once.

5. **Provenance everywhere, or it didn't happen.**
   No source span, no claim - the line between a demo and a system you can ship.

The model is a component. The architecture is the product.

Getting Started

Minimal stack, reading list, traps to avoid.

MINIMAL STACK TO REPRODUCE

- **Docling** - PDF → clean markdown
- **GLiNER + one LLM** - entities, relations, verbalization
- **Neo4j Community** - graph + vectors — free
- **a small reranker** - bge-reranker — cheap precision
- **one LLM API** - extraction & answers

READING LIST

- → **Microsoft GraphRAG** - the local-vs-global paper
- → **Neo4j GraphRAG package** - docs + working examples
- → **GLiNER paper** - zero-shot NER, one pass
- → **schema.org / GoodRelations** - ontology inspiration
- → **Neuro-symbolic survey** - the bigger picture

PITFALLS

- ✗ **VLM silent hallucination** — it invents cells — verify tables
- ✗ **Entity resolution neglect** — the graph splits your entities
- ✗ **Over-granular ontologies** — too fine → nothing links up
- ✗ **A vector DB you don't need** — pgvector / Neo4j already cover it

THANKS!